

Advanced and Distributed Algorithms

Geometric algorithms

Dr. Benoît PIRANDA

Ph.D. Associate Professor HDR

Member of Computer Science Department of Femto-ST institute

benoit.piranda@femto-st.fr



Outline



- 2D Algorithmic
 - Convexity and non-convexity
 - Convex hull
 - Delaunay triangulation
 - Creating a regular mesh for a set of points
 - Voronoï diagram
- 3D Algorithmic
 - Geometric modeling using CSG trees
 - Object picking / ray-tracing
 - Z-Buffer algorithm, OpenGL library

Basic 2D functions

- Distance between two points

- Distance from A to B

$$d(A, B) = \left\| \overrightarrow{AB} \right\| = \sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}$$

- Orthogonal direction in 2D

- $\vec{v}$ Orthogonal direction (to the right) of $\vec{u}$

$$\vec{v} = \vec{u} \wedge \vec{z} = \begin{pmatrix} u_x \\ u_y \\ 0 \end{pmatrix} \wedge \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} u_y \\ -u_x \\ 0 \end{pmatrix}$$

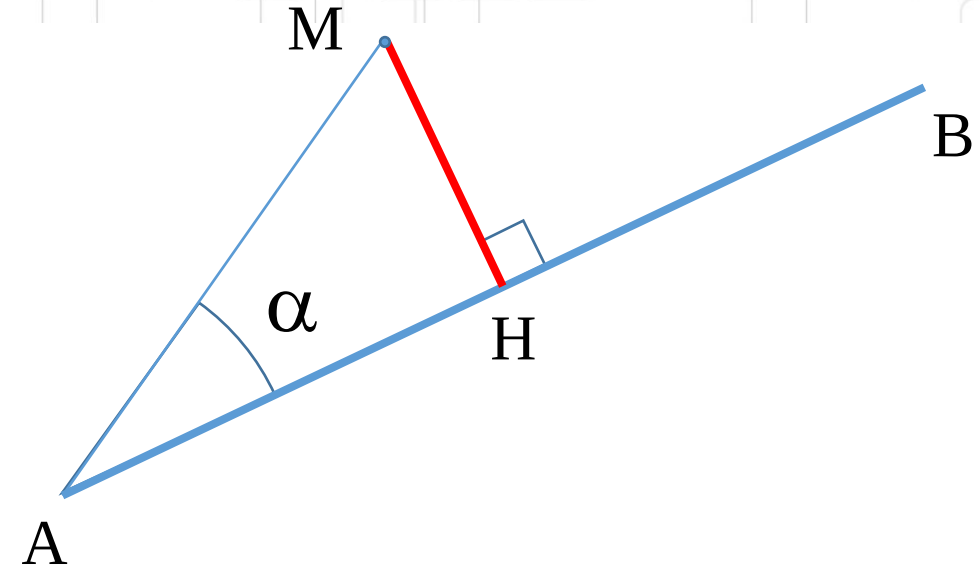
Basic 2D functions

- Distance from a point M to a line (AB):

$$\overrightarrow{AB} \times \overrightarrow{AM} = \sin \alpha \times \|\overrightarrow{AB}\| \times \|\overrightarrow{AM}\| \times \vec{z}$$

$$\|\overrightarrow{MH}\| = \sin \alpha \times \|\overrightarrow{AM}\| = \frac{\|\overrightarrow{AB} \times \overrightarrow{AM}\|}{\|\overrightarrow{AB}\|}$$

$$\overrightarrow{AB} \times \overrightarrow{AM} = \begin{pmatrix} 0 \\ 0 \\ AB_x AM_y - AB_y AM_x \end{pmatrix}$$



$$\|\overrightarrow{MH}\| = \frac{AB_x AM_y - AB_y AM_x}{\sqrt{AB_x^2 + AB_y^2}}$$

Basic 2D functions

- Distance to a segment: $H \in [AB]$
 - Compute the position of H in [AB]

$$\begin{aligned}\vec{AB} \cdot \vec{AM} &= \cos \alpha \times \|\vec{AB}\| \times \|\vec{AM}\| \\ \|\vec{AH}\| &= \cos \alpha \times \|\vec{AM}\| = \frac{\|\vec{AB} \cdot \vec{AM}\|}{\|\vec{AB}\|}\end{aligned}$$



$$\|\vec{AH}\| = \frac{AB_x AM_x + AB_y AM_y}{\sqrt{AB_x^2 + AB_y^2}}$$

```
QPair<double,Vector2D> distanceToEdge(const Vector2D &M, int i) {  
    Vector2D AB = tabPts[i+1]-tabPts[i];  
    Vector2D AM = M-tabPts[i];  
    double lAB = AB.length();  
    double AH = (AB.x*AM.x + AB.y*AM.y)/lAB;  
    if (AH<0) return QPair<double,Vector2D>(AM.length(),tabPts[i]);  
    else if (AH>lAB) return QPair<double,Vector2D>((M-tabPts[i+1]).length(),tabPts[i+1]);  
    else {  
        Vector2D MH=tabPts[i]+AB*(AH/lAB)-M;  
        return QPair<double,Vector2D>(MH.length(),M+MH);  
    }  
}
```



Geometric algorithms on polygons

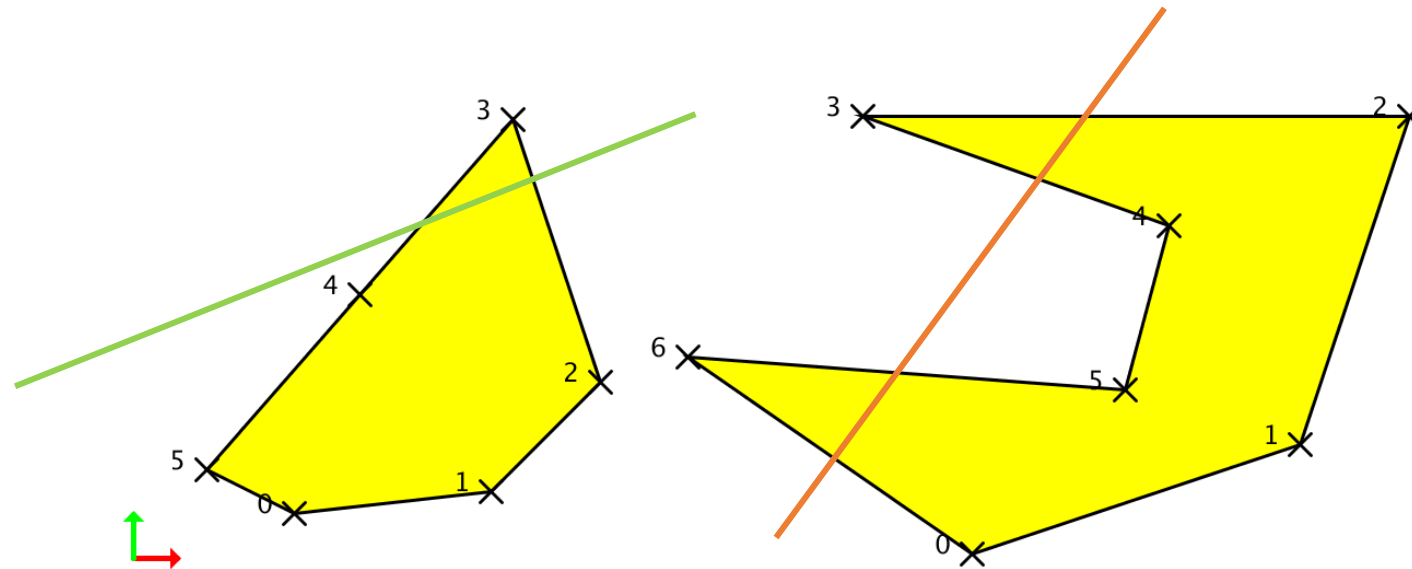
Definitions



- Definitions
 - **A polygon is a plane figure** that is described by a finite number of straight line segments connected to form a closed polygonal chain or polygonal circuit.
 - The segments of a polygonal circuit are called its **edges** or **sides**, and the points where two edges meet are the polygon's **vertices** or corners.
 - A **simple polygon** is one which does not intersect itself.

Convexity and non-convexity

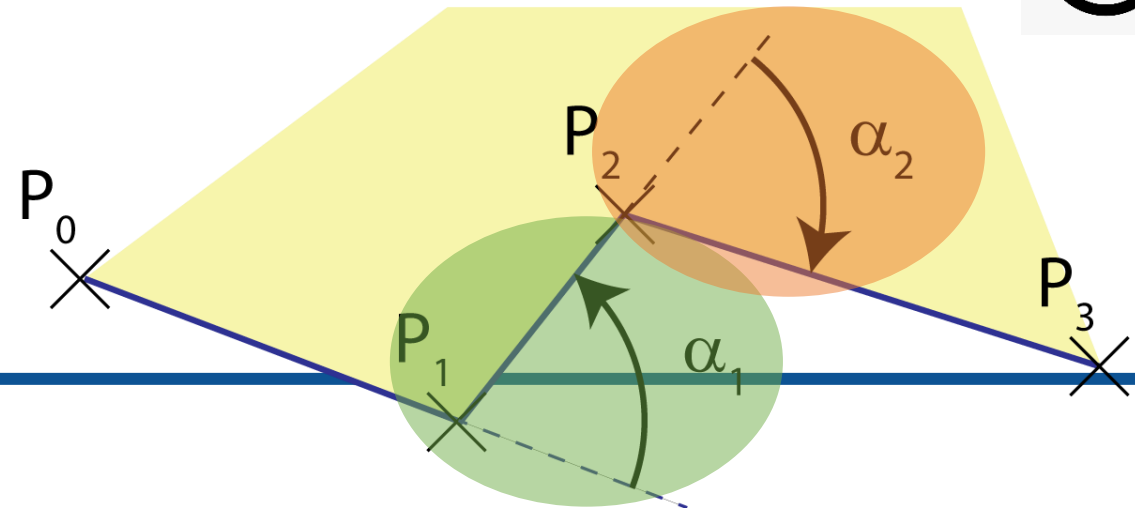
- Propriety #1
 - A simple polygon is **convex** if **any line** drawn through the polygon (and not tangent to an edge or corner) meets its boundary **exactly twice**.



Convexity and non-convexity

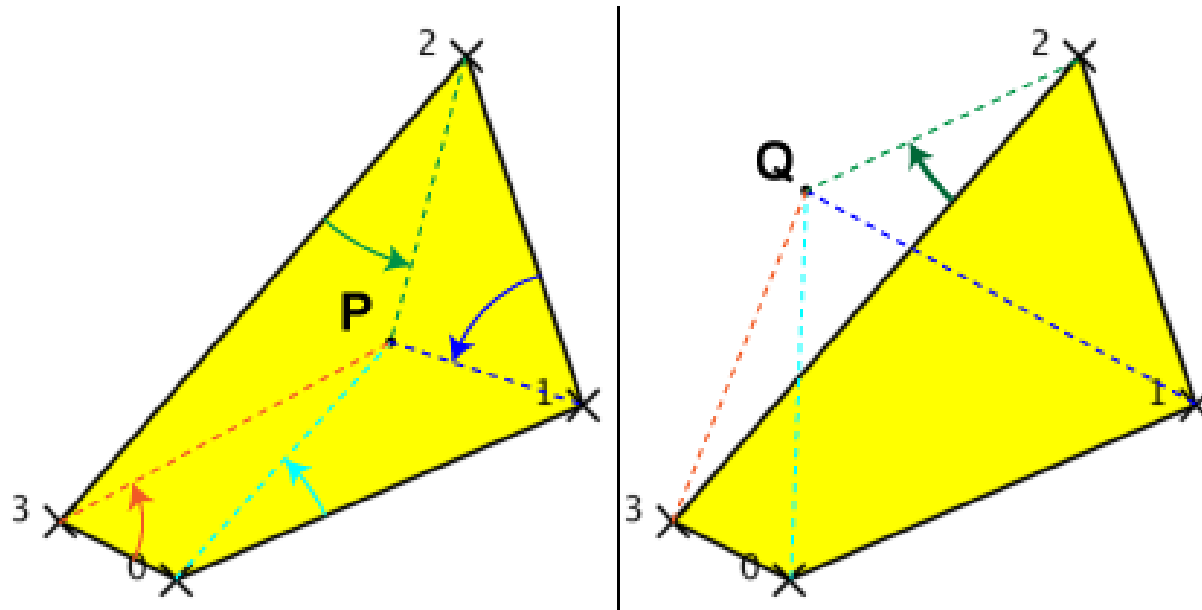
- Propriety #2
 - A polygon is concave if there is at least one interior angle greater than 180° .
 - It exists $\alpha_i = \overrightarrow{P_{i-1}P_i} \widehat{\overrightarrow{P_iP_{i+1}}} < 0$
- Convexity algorithm:
 - We define a polygon by a list of N 2D vertices $P_i \mid i \in [0..N - 1]$ ordered counterclockwise.

$$\text{sign}(\alpha_i) = \text{sign} \left(\left[\overrightarrow{P_{i-1}P_i} \wedge \overrightarrow{P_iP_{i+1}} \right]_z \right)$$



Position of point / a convex polygon

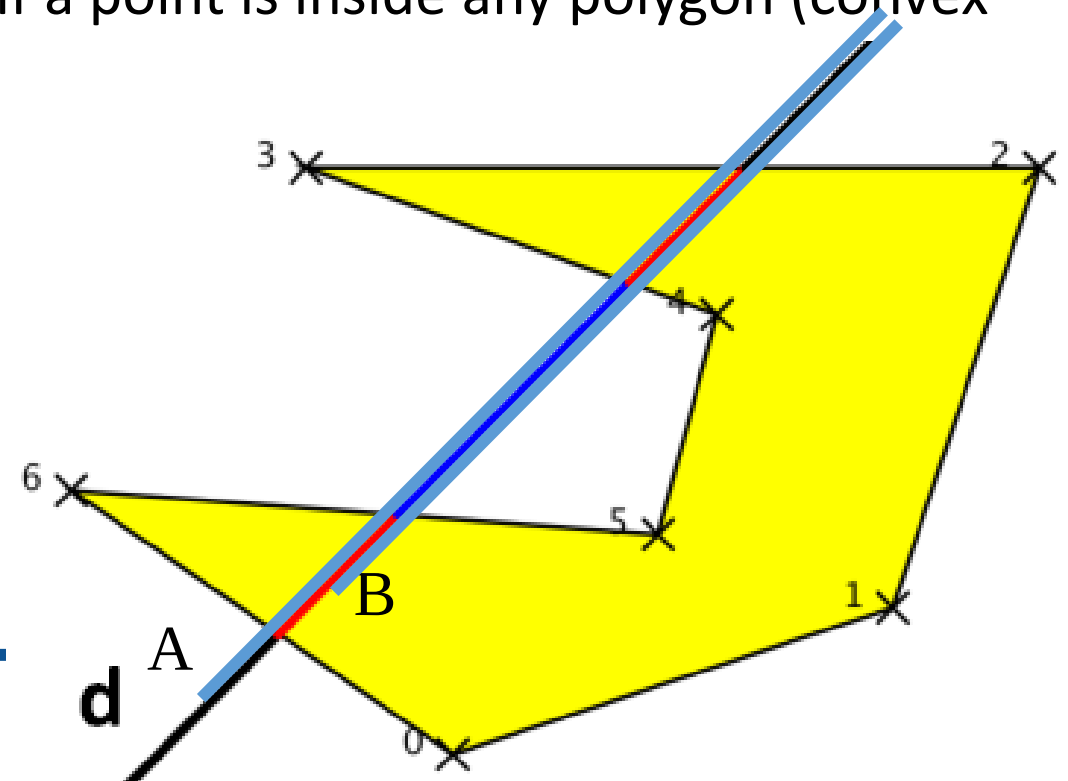
- Due to counterclockwise order of the vertices, a point P is in the **interior of a convex polygon** if P is on the left of every edges:



$$P_i P_{i+1} \mid i \in [0..N]$$

Position of point / a concave polygon

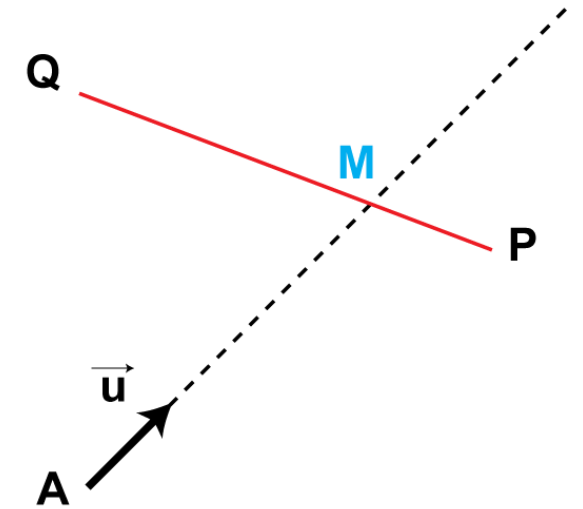
- Calculation of the intersection points between a line and a polygon
 - How many points between a line and a polygon may we get?
 - Demonstrate that this number is even!
 - Deduce an algorithm allowing to check if a point is inside any polygon (convex or concave)!
 - Complexity? $O(N)$



Intersection between a ray and a segment

- Let $[A\vec{u})$ be a ray (half-line), $[PQ]$ a segment, we search the point $M = [A\vec{u}) \cap [PQ]$

$$\begin{cases} \overrightarrow{AM} = k_1 \vec{u} \\ k_1 \geq 0 \\ \overrightarrow{PM} = k_2 \overrightarrow{PQ} \\ 0 \leq k_2 \leq 1 \end{cases} \quad \longrightarrow \quad \begin{cases} \overrightarrow{PA} + k_1 \vec{u} = k_2 \overrightarrow{PQ} \\ k_1 \geq 0 \\ 0 \leq k_2 \leq 1 \end{cases}$$



We deduce from this system, 2 equations and 2 unknowns:

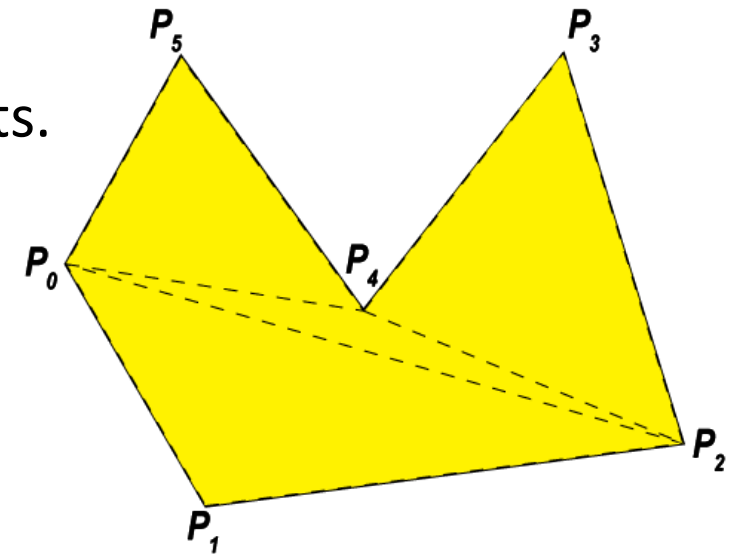
$$\begin{pmatrix} u_x & -PQ_x \\ u_y & -PQ_y \end{pmatrix} \begin{pmatrix} k_1 \\ k_2 \end{pmatrix} = \begin{pmatrix} AP_x \\ AP_y \end{pmatrix} \quad \longrightarrow \quad \begin{pmatrix} k_1 \\ k_2 \end{pmatrix} = \begin{pmatrix} u_x & -PQ_x \\ u_y & -PQ_y \end{pmatrix}^{-1} \begin{pmatrix} AP_x \\ AP_y \end{pmatrix}$$

Intersection between a ray and a segment

- We get:
$$\left\{ \begin{array}{l} k_1 = \frac{AP_x PQ_y - AP_y PQ_x}{u_x PQ_y - u_y PQ_x} \\ k_1 \geq 0 \\ k_2 = \frac{AP_x u_y - AP_y u_x}{u_x PQ_y - u_y PQ_x} \\ 0 \leq k_2 \leq 1 \end{array} \right.$$
- A is in the border if:
 - $\exists i \mid (A = P_i)$
 - $(AP_x PQ_y - AP_y PQ_x = 0) \wedge (0 \leq AP_x u_y - AP_y u_x \leq u_x PQ_y - u_y PQ_x)$
- Non collinearity condition $u_x PQ_y - u_y PQ_x \neq 0$
 - Warning: if $[A\vec{u})$ is collinear to an edge $[P_i P_{i+1}]$, there is no intersection with this edge.

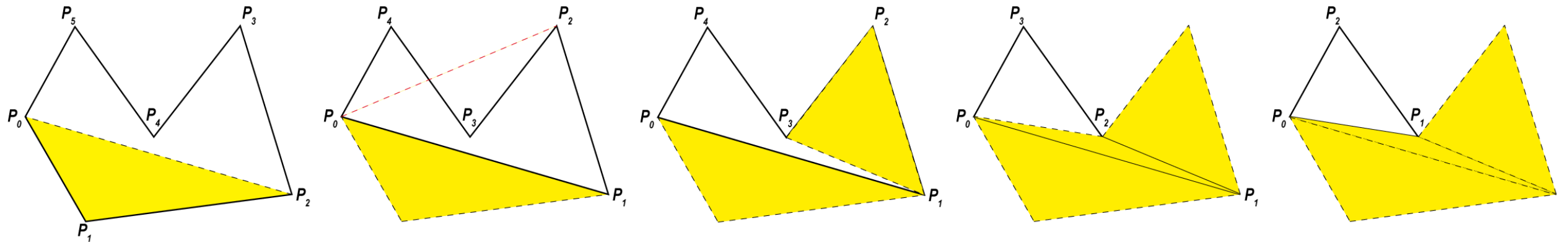
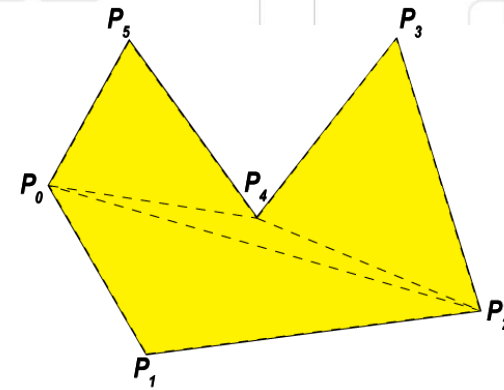
Simple triangulation algorithms

- Goal: transform a concave polygon into a set of convex polygons
- Ear clipping method:
 1. Searching a triangle $(P_i P_{i+1} P_{i+2})$ which does not contains any other point of the polygon.
 2. Remove the point P_{i+1} from the polygon.
 3. Steps 1 and 2 are repeated until it remains only 2 points.



Ear clipping algorithm

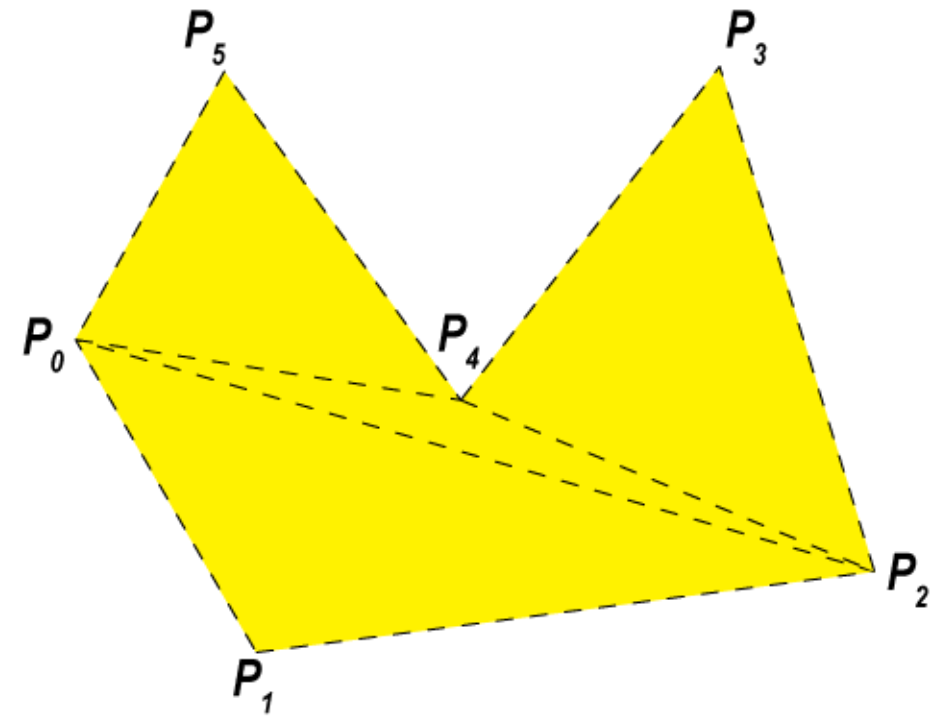
1. Searching a triangle $(P_i P_{i+1} P_{i+2})$ which does not contain any other point of the polygon, $(i \in [0..n - 2])$.
2. Remove the point P_{i+1} from the polygon (indices).
3. Steps 1 and 2 are repeated until it remains only 2 points.



Compute the surface of a polygon

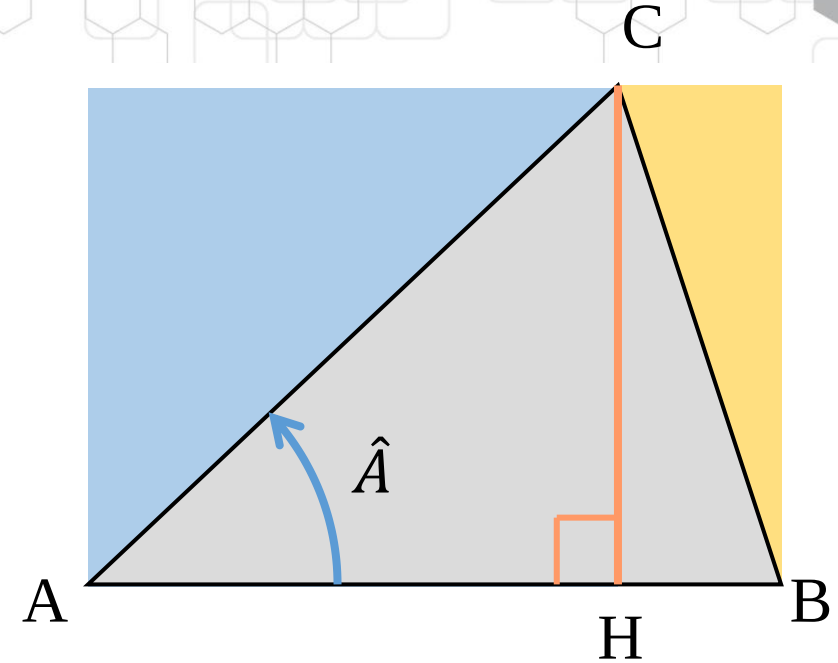
- How to compute the surface of a polygon
 1. Triangulate into $T = \{T_0, T_1, \dots, T_{n-1}\}$
 2. Sum up the surfaces of the triangles

$$S = \sum_{i=0}^{n-1} S(T_i)$$



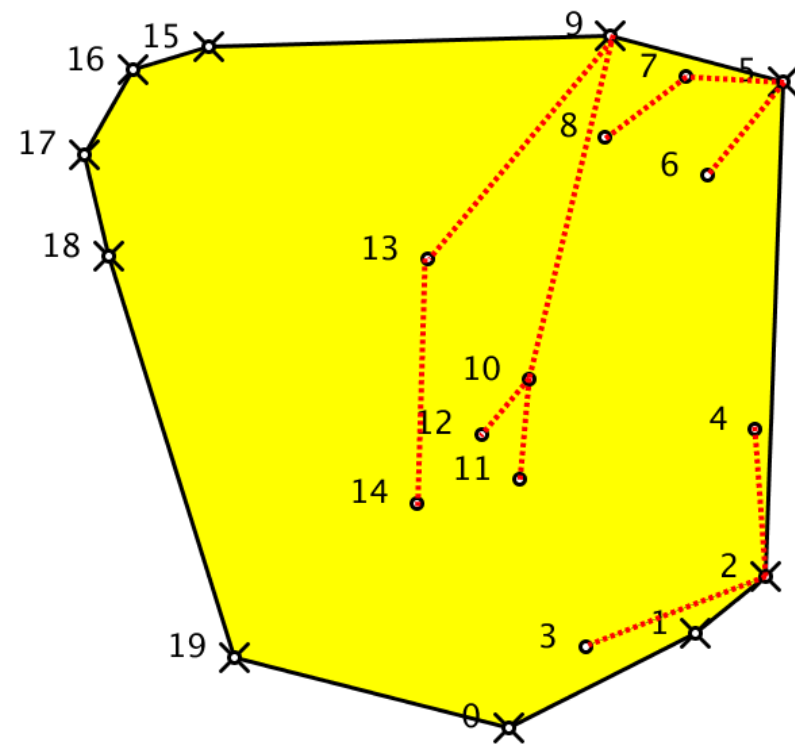
Compute the surface of simple triangle

- $S_{(ABC)} = S_{(AHC)} + S_{(HBC)}$
- $S_{(AHC)} = \frac{1}{2} AH \times HC$ and $S_{(HBC)} = \frac{1}{2} HB \times HC$
- $S_{(ABC)} = \frac{1}{2} (AH + HB) \times HC$
- $HC = \sin \hat{A} \times AC$
- $S_{(ABC)} = \frac{1}{2} (\sin \hat{A} \times AB \times AC)$
- $S_{(ABC)} = \frac{1}{2} [\overrightarrow{AB} \times \overrightarrow{AC}]_z$



$$S_{(ABC)} = \frac{\overrightarrow{AB}_x \times \overrightarrow{AC}_y - \overrightarrow{AB}_y \times \overrightarrow{AC}_x}{2}$$

Convex hull

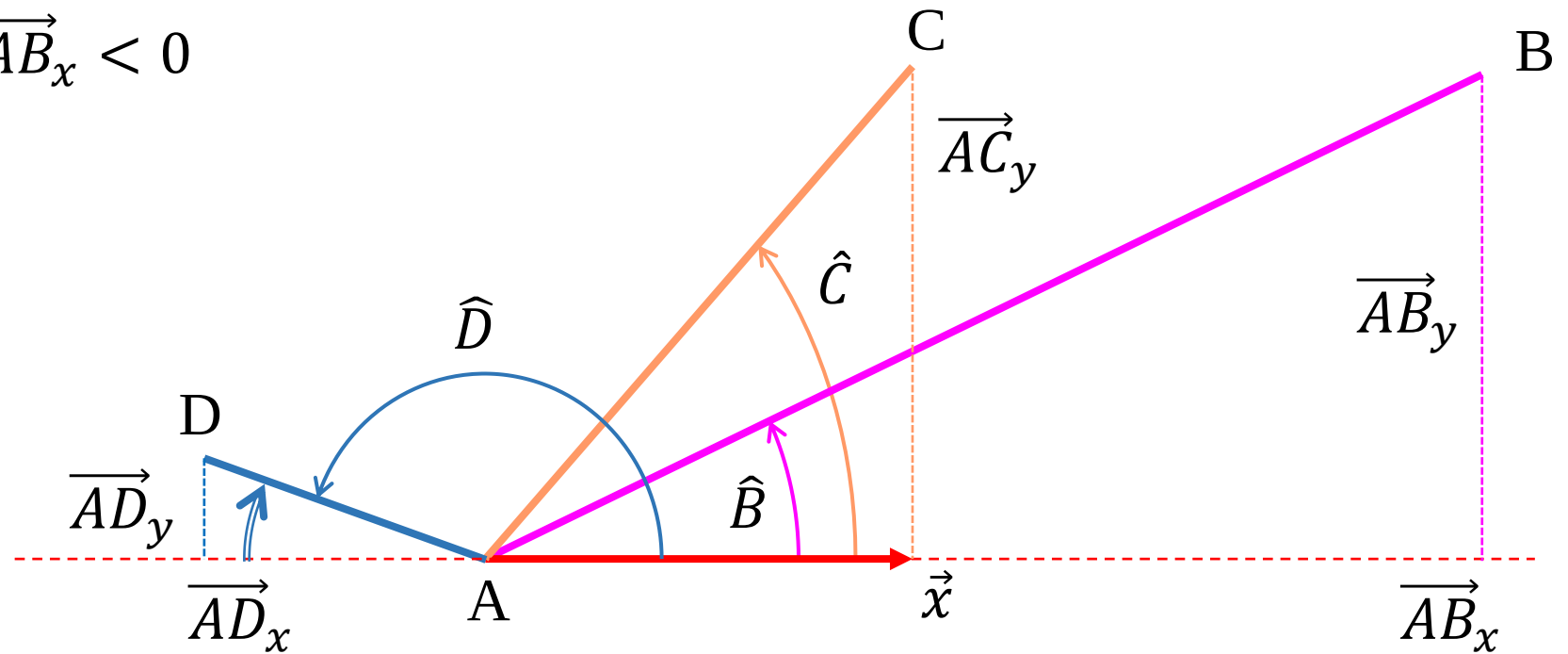


Convexe hull

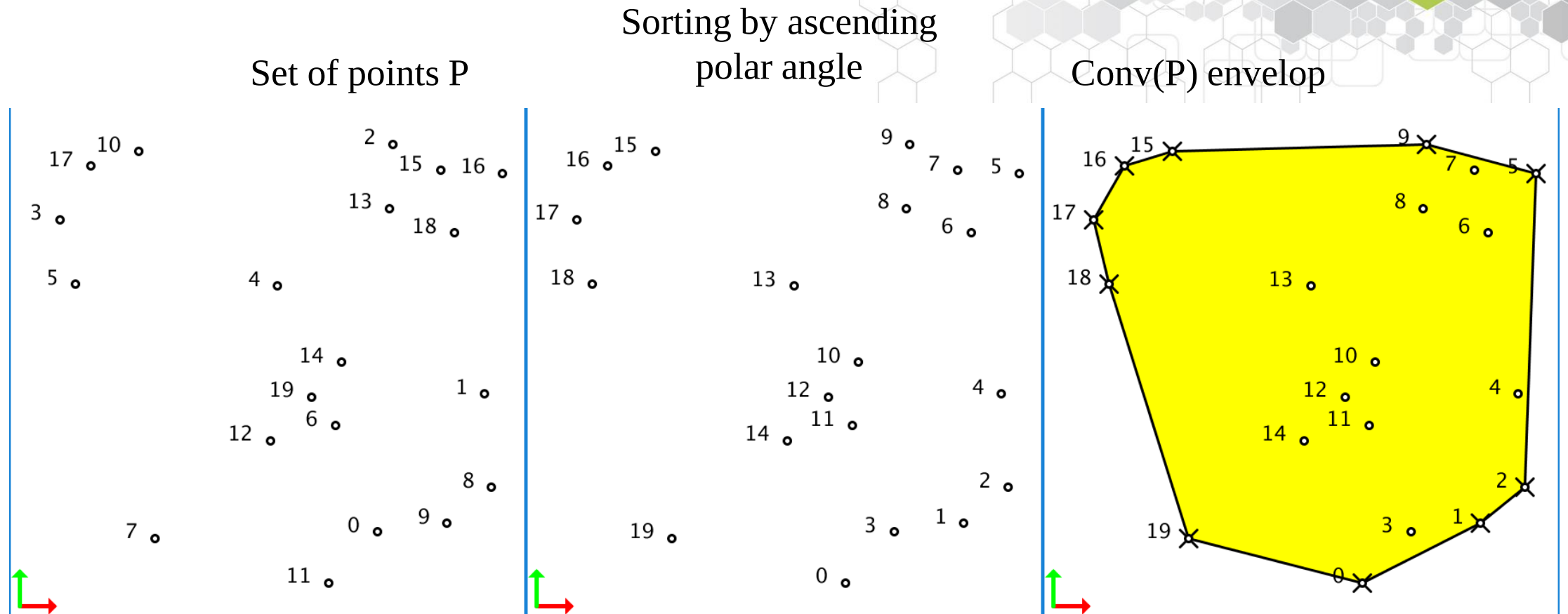
- Definition:
 - The **convex hull** (or **convex envelop**) of a set of point P_i of $\mathcal{R}^2$ notated $Conv(P)$ is the smallest convex polygon S which admits every points P_i in its border or inside.
- Graham scan algorithm:

Ascending polar angle

$$\hat{B} = \begin{cases} \sin^{-1} \left(\frac{\overrightarrow{AB}_y}{AB} \right) & \overrightarrow{AB}_x \geq 0 \\ \pi - \sin^{-1} \left(\frac{\overrightarrow{AB}_y}{AB} \right) & \overrightarrow{AB}_x < 0 \end{cases}$$

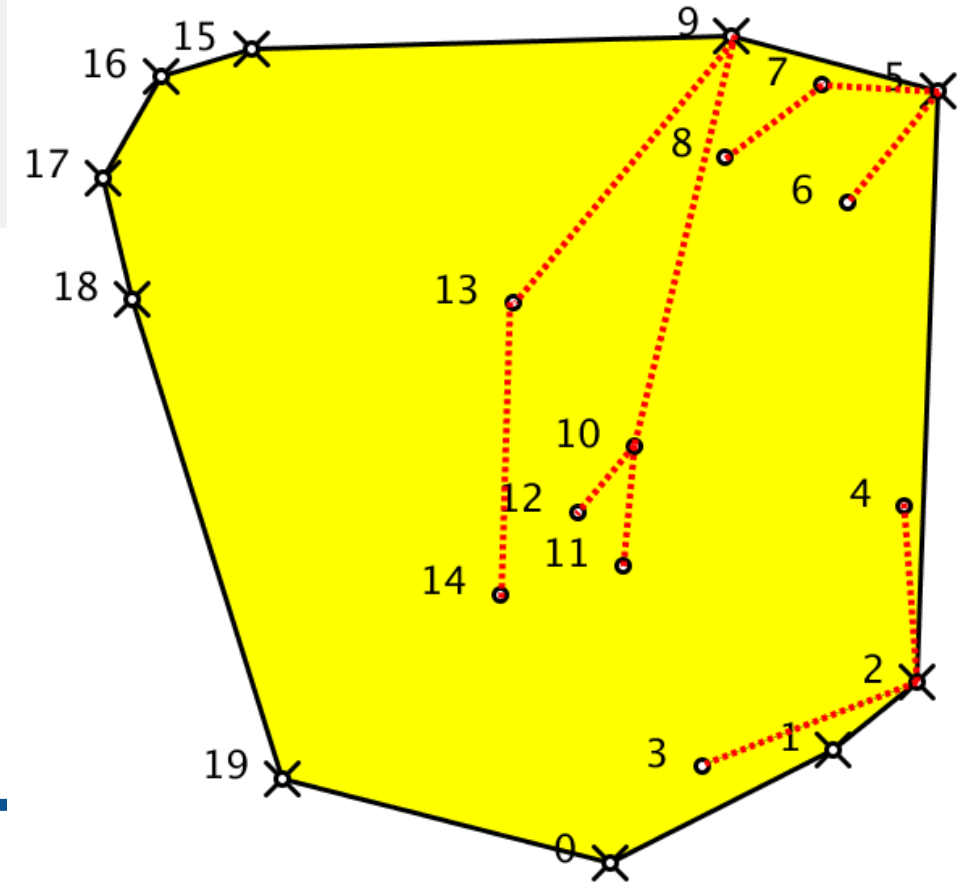


Exemple

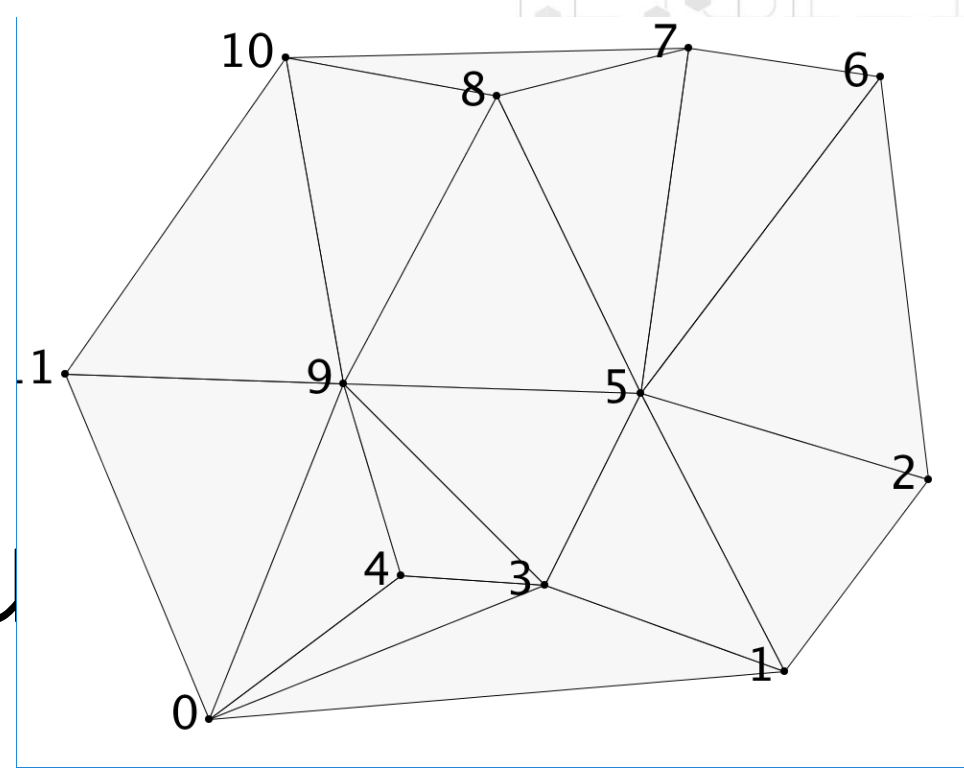


Algorithm steps

```
S.push({ $P_0$ ,  $P_1$ ,  $P_2$ })  
for i = 3 to N-1 do  
  while (CCW(S.top(-1), S.top(),  $P_i$ )  $\leq 0$ ) do  
    S.pop()  
  done  
  S.push( $P_i$ )  
done  
Create Conv using S points
```



Delaunay triangulation



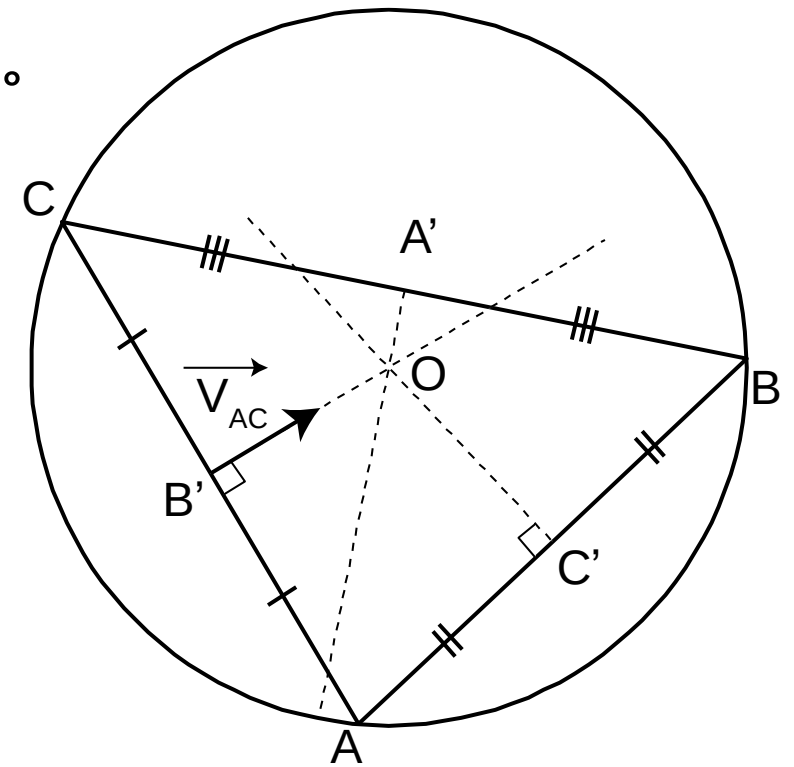
Delaunay triangulation

- Definitions

- The **triangulation** of a set of points consists in building a set of disjoint triangles which fully **cover the convex hull** of the set of points. This set of triangles defines a **mesh**.
- One important problem is to find a mesh as regular as possible, i.e. made of large triangles, which shape is close to equilateral triangle.
- Delaunay's criteria:
The **Delaunay triangulation** (also known as a Delone triangulation) for a given set P of discrete points in a plane is a triangulation $DT(P)$ such that **no point in P is inside the circumcircle of any triangle in $DT(P)$** .

Circumscribed circle or Circumcircle

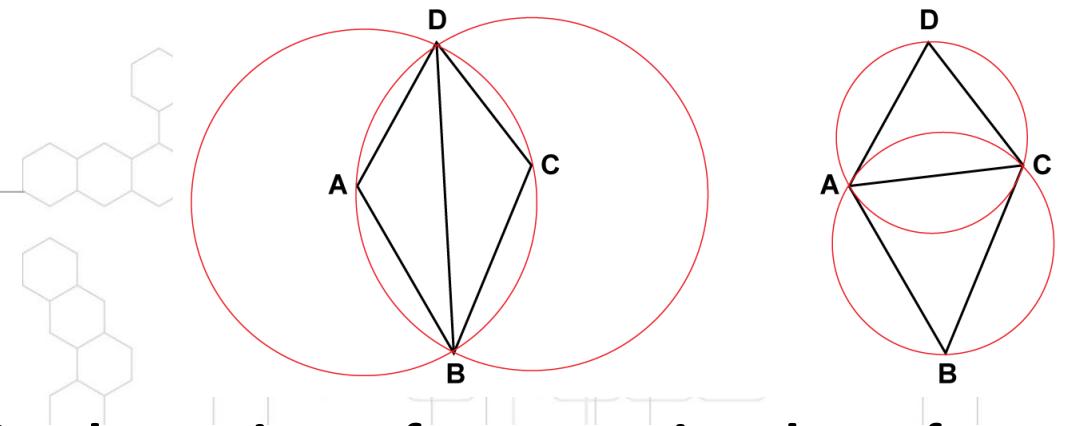
- The circumcenter of a triangle is the intersection of the 3 perpendicular bisectors.
 - The line which cut an edge into two equal parts at 90°



Delaunay triangulation

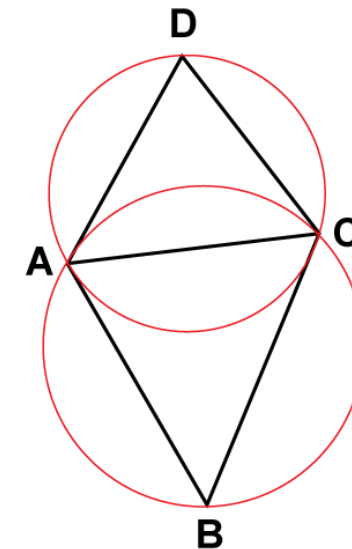
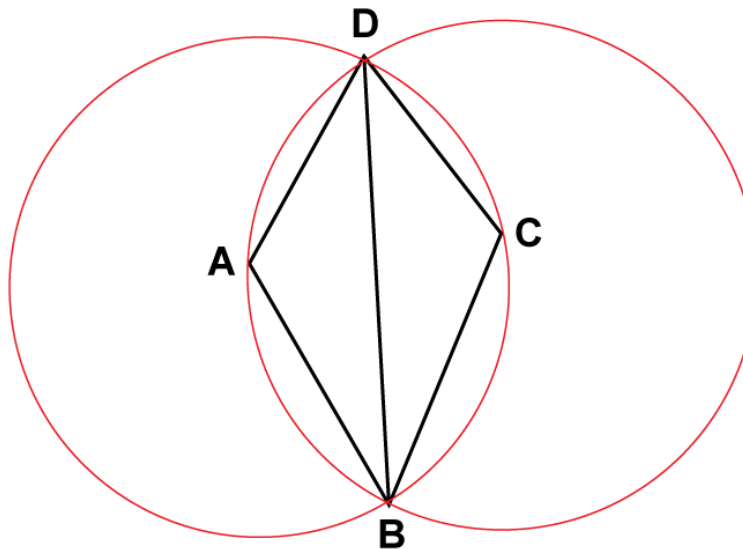
- Theorems

- Let N be the number of points of P . **The union of every triangles of a triangulation makes the convex hull of the points.**
- In the $\mathcal{R}^2$ plane, the triangulation of Delaunay is made of $O(N)$ triangles, if the convex hull is made of b vertices, each possible triangulation admits exactly $2N - 2 - b$ triangles.
- If two points P_i and P_j are on the border of a circle C and there is **no other point** P_k inside C , then the edge $[P_i P_j]$ is part of the Delaunay triangulation.
- “Opposite angle propriety”: Considering two given triangles ABC and ACD , the sum of the angles $\hat{B}$ and $\hat{D}$ is less or equal to 180° if and only if these triangles follows Delaunay’s criteria.



Delaunay triangulation algorithm

- Delaunay triangulation algorithm :
 - If two triangles ABD and BCD do not respect the Delaunay's criteria
 - We flip the common edge $[BD]$ to $[AC]$ in order to replace the two previous triangles by ABC and ACD which respect the Delaunay's criteria.



Check if a point D is in a circumcircle of ABC

- Compute the center O and the radius r and compare OD to r

$$\overrightarrow{AO} = \overrightarrow{AB'} + \overrightarrow{B'O}$$

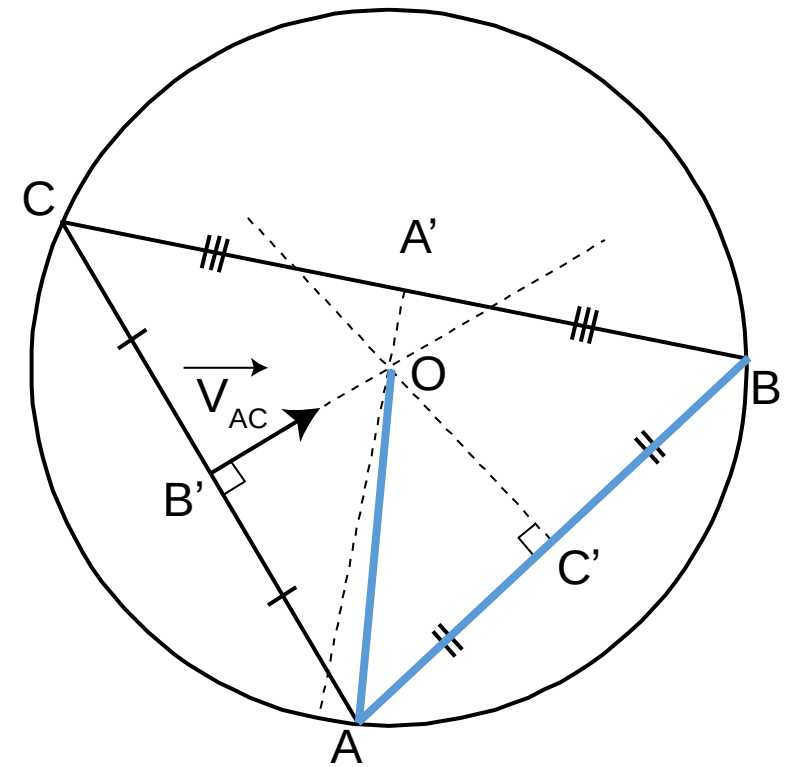
$$\overrightarrow{B'O} = k \times \overrightarrow{V_{AC}} = k \times \text{ortho}(AC)$$

$$\overrightarrow{AO} \cdot \overrightarrow{AB} = (\overrightarrow{AB'} + \overrightarrow{B'O}) \cdot \overrightarrow{AB}$$

$$\overrightarrow{AO} \cdot \overrightarrow{AB} = AO \times AB \times \cos(\hat{A}) = AC \times AB \times \cos(\hat{A}) = \frac{1}{2} AB^2$$

$$\overrightarrow{AO} \cdot \overrightarrow{AB} = \left(\frac{1}{2} \overrightarrow{AC} + k \times \overrightarrow{V_{AC}} \right) \cdot \overrightarrow{AB} = \frac{1}{2} AB^2$$

$$k = \frac{AB^2 - \overrightarrow{AC} \cdot \overrightarrow{AB}}{2 \times \overrightarrow{V_{AC}} \cdot \overrightarrow{AB}}$$



k is the abscissa of O in $[B' \overrightarrow{V_{AC}})$

Check if a point D is in a circumcircle of ABC

- Theorem:

- In $\mathbb{R}^2$, the point D is inside the circumcircle of the triangle ABC if the determinant of the following matrix is positive.

$$\det \begin{pmatrix} A_x & A_y & A_x^2 + A_y^2 & 1 \\ B_x & B_y & B_x^2 + B_y^2 & 1 \\ C_x & C_y & C_x^2 + C_y^2 & 1 \\ D_x & D_y & D_x^2 + D_y^2 & 1 \end{pmatrix} > 0 \iff \det \begin{pmatrix} A_x - D_x & A_y - D_y & A_x^2 - D_x^2 + A_y^2 - D_y^2 \\ B_x - D_x & B_y - D_y & B_x^2 - D_x^2 + B_y^2 - D_y^2 \\ C_x - D_x & C_y - D_y & C_x^2 - D_x^2 + C_y^2 - D_y^2 \end{pmatrix} > 0$$

About determinant of matrices

$$\det \begin{pmatrix} a & c \\ b & d \end{pmatrix} = ad - bc$$

$$\det \begin{pmatrix} a & d & g \\ b & e & h \\ c & f & i \end{pmatrix} = a \times \det \begin{pmatrix} e & h \\ f & i \end{pmatrix} - b \times \det \begin{pmatrix} d & g \\ f & i \end{pmatrix} + c \times \det \begin{pmatrix} d & g \\ e & h \end{pmatrix}$$

Comparison of two methods

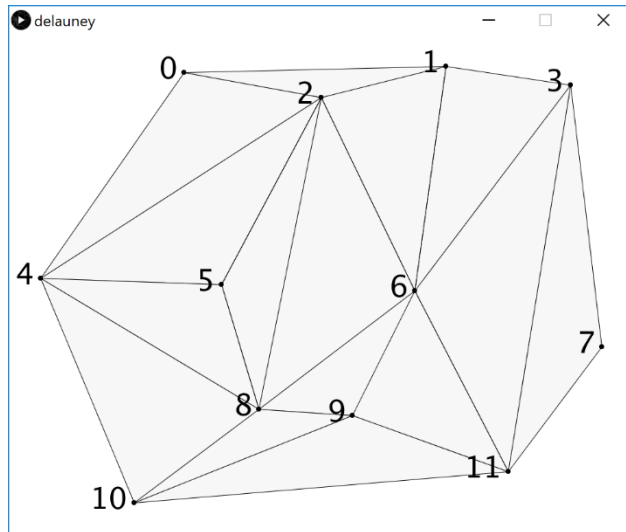
- Advantage of the second method
 - Is D is A,B or C

$$\det \begin{pmatrix} A_x - D_x & A_y - D_y & A_x^2 - D_x^2 + A_y^2 - D_y^2 \\ B_x - D_x & B_y - D_y & B_x^2 - D_x^2 + B_y^2 - D_y^2 \\ C_x - D_x & C_y - D_y & C_x^2 - D_x^2 + C_y^2 - D_y^2 \end{pmatrix}$$

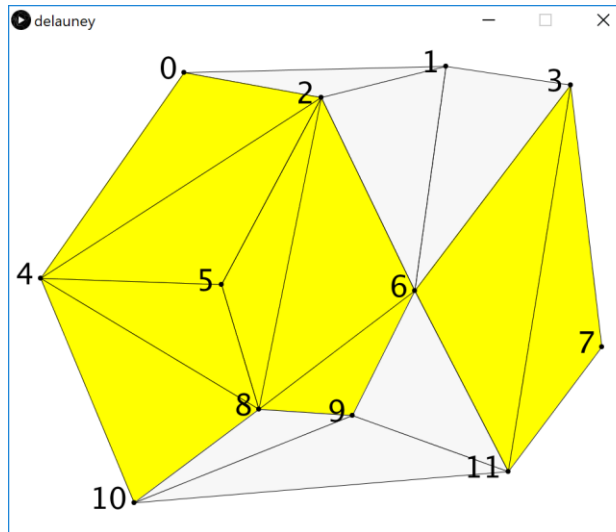
$$\det \begin{pmatrix} 0 & 0 & 0 \\ B_x - D_x & B_y - D_y & B_x^2 - D_x^2 + B_y^2 - D_y^2 \\ C_x - D_x & C_y - D_y & C_x^2 - D_x^2 + C_y^2 - D_y^2 \end{pmatrix} = 0$$

Delaunay triangulation algorithm

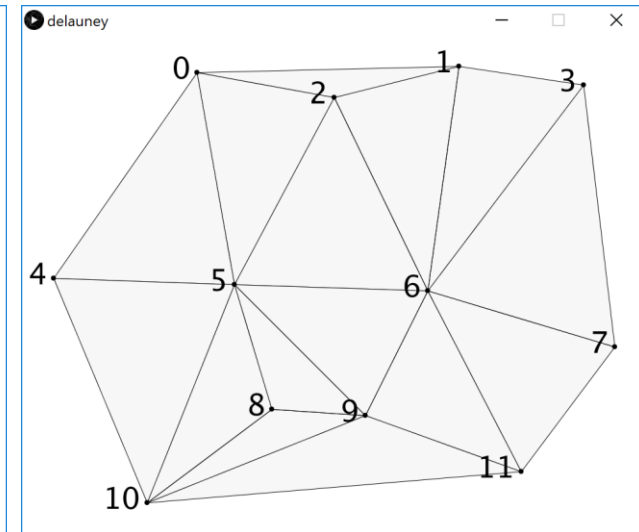
- Example on a given mesh:



Initial mesh

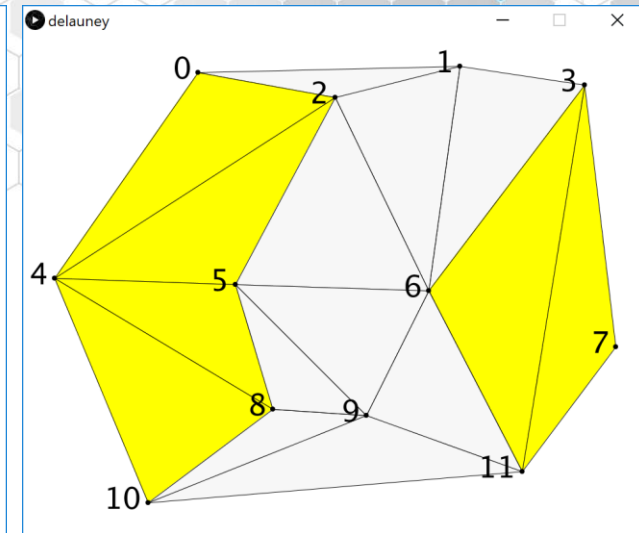
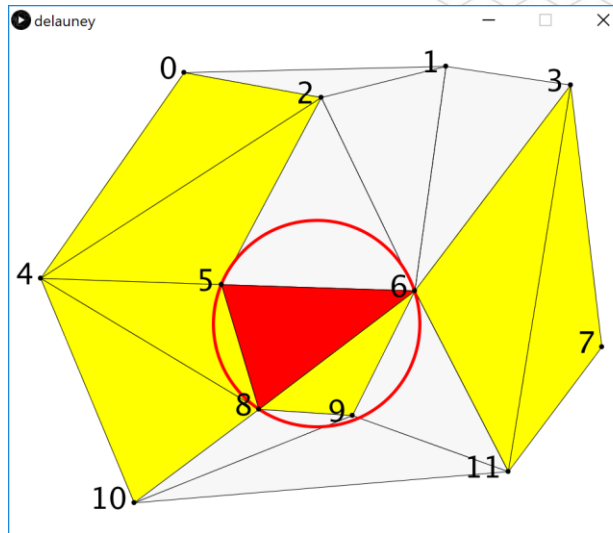
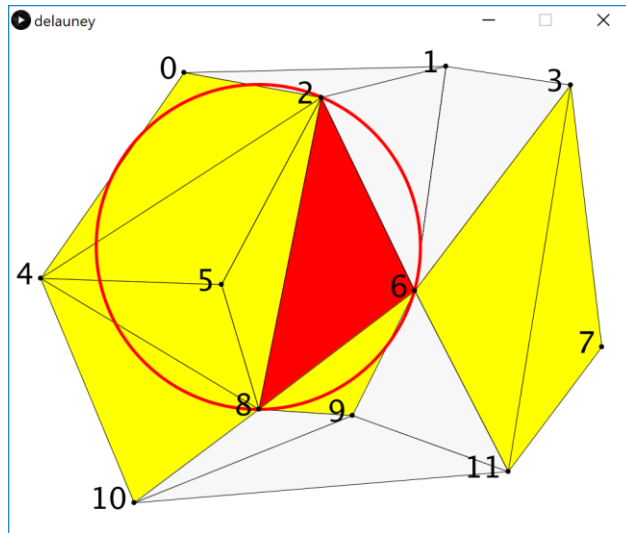


Non Delaunay
triangles in yellow

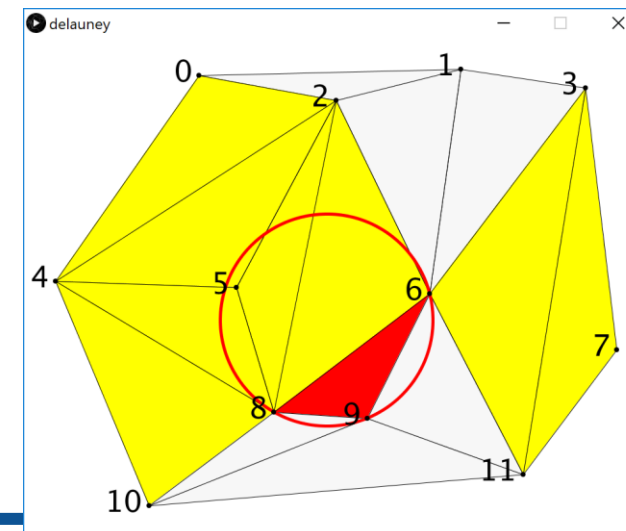


After flips

Flip orders!

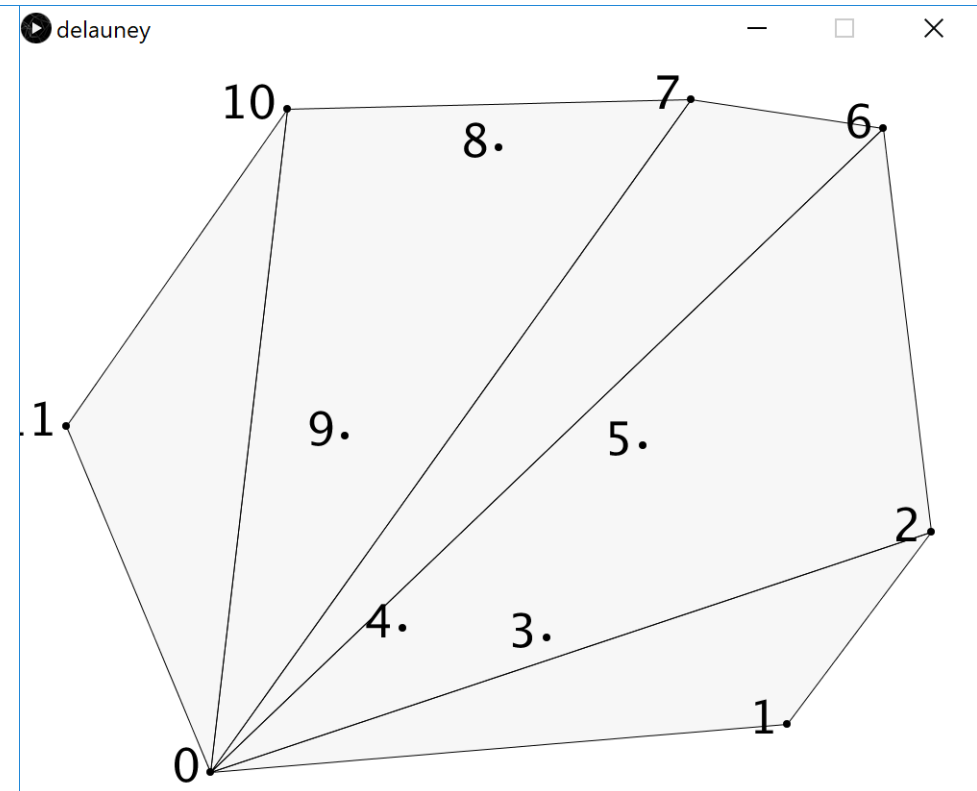
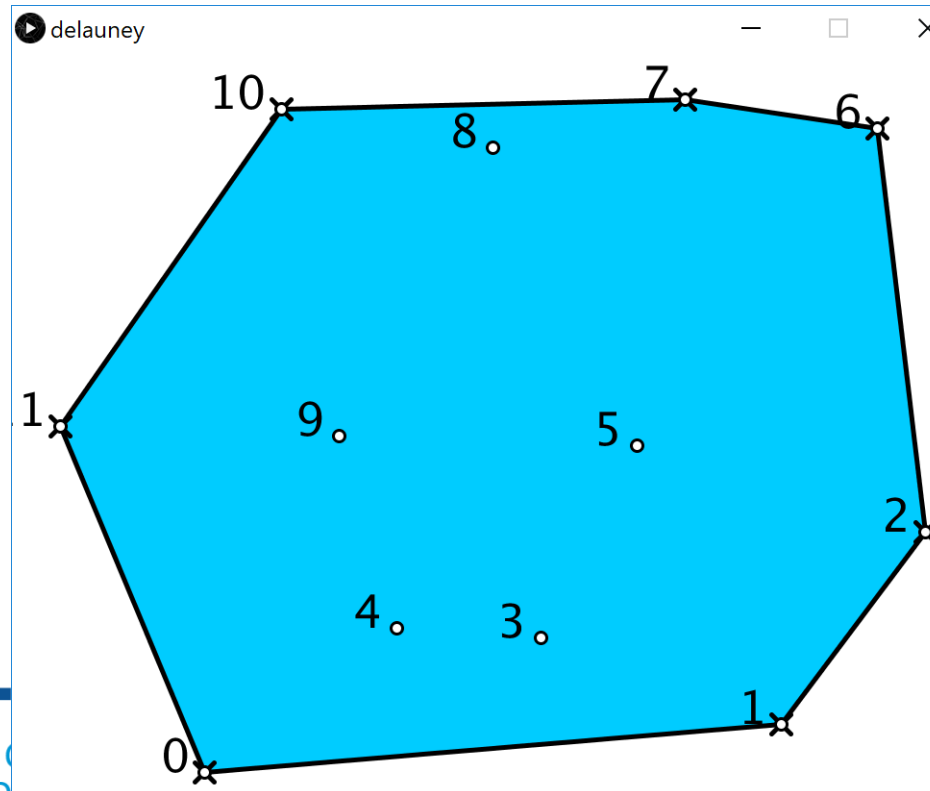


- The flipping is impossible if the point which is inside the circumcircle is not a vertex of a neighbor triangle.
- This flipping of the triangle is postponed to the end of the treatments.



Creating a mesh from a set of points

- 1 Calculate **convex hull** of points $P(P_0 \dots P_{N-1})$
- 2 **Triangulation** of the convex hull $E(E_0 \dots E_{m-1})$,
 - Create a $\text{Triangle}(E_0, E_i, E_{i+1}) \forall i \in [1, m-2]$



Creating a mesh from a set of points

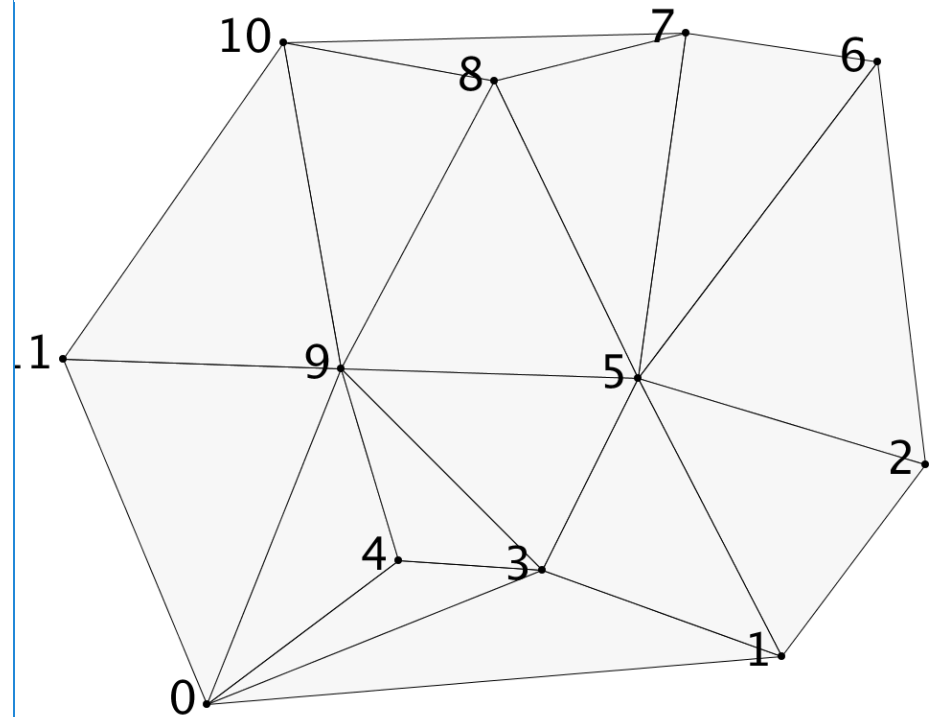
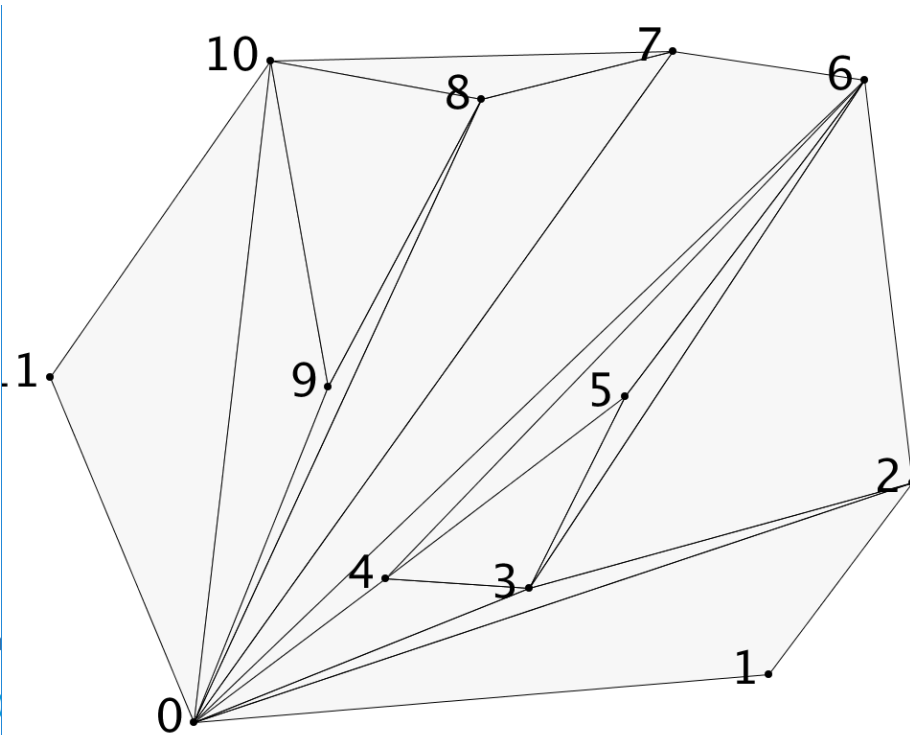
3 • Add the **interior points**

- If $P_i \in T(P_A, P_B, P_C)$ then

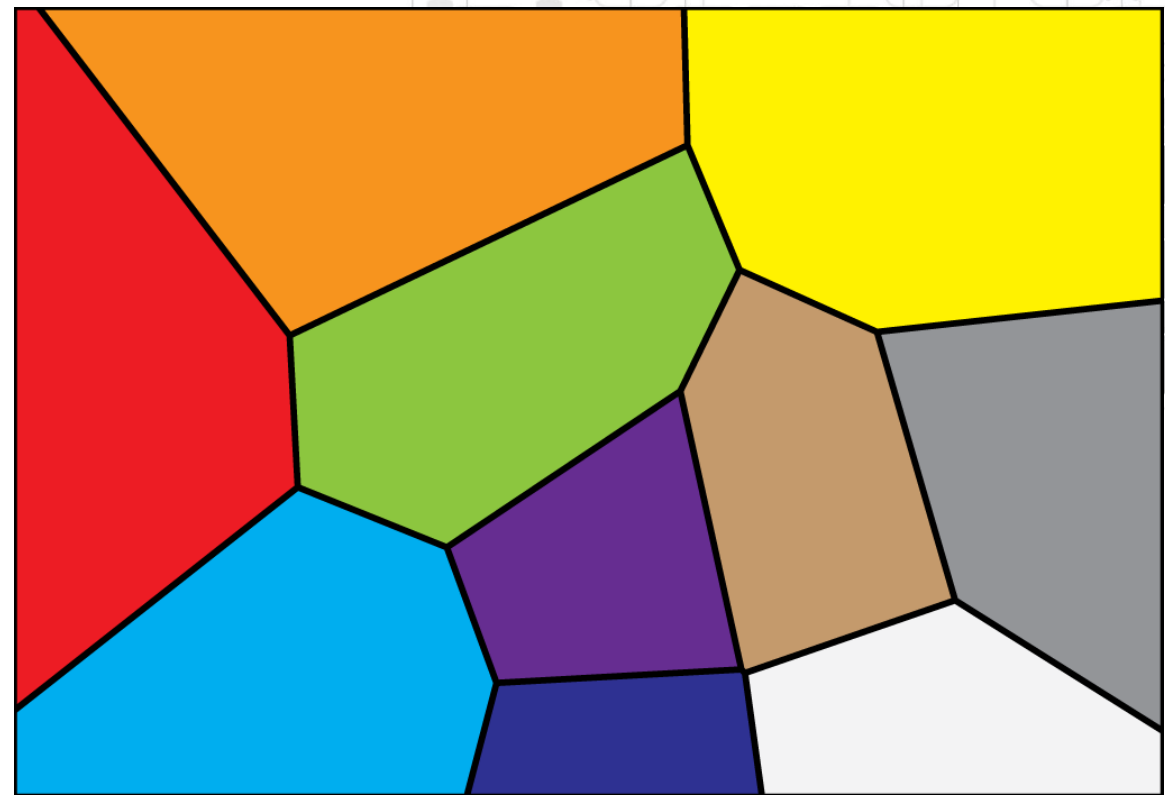
- delete T

- and add the triangles (P_A, P_B, P_i) , (P_B, P_C, P_i) and (P_C, P_A, P_i)

4 • Apply the Delaunay triangulation algorithm.



Voronoi Diagram



Voronoi Diagram

- Definitions

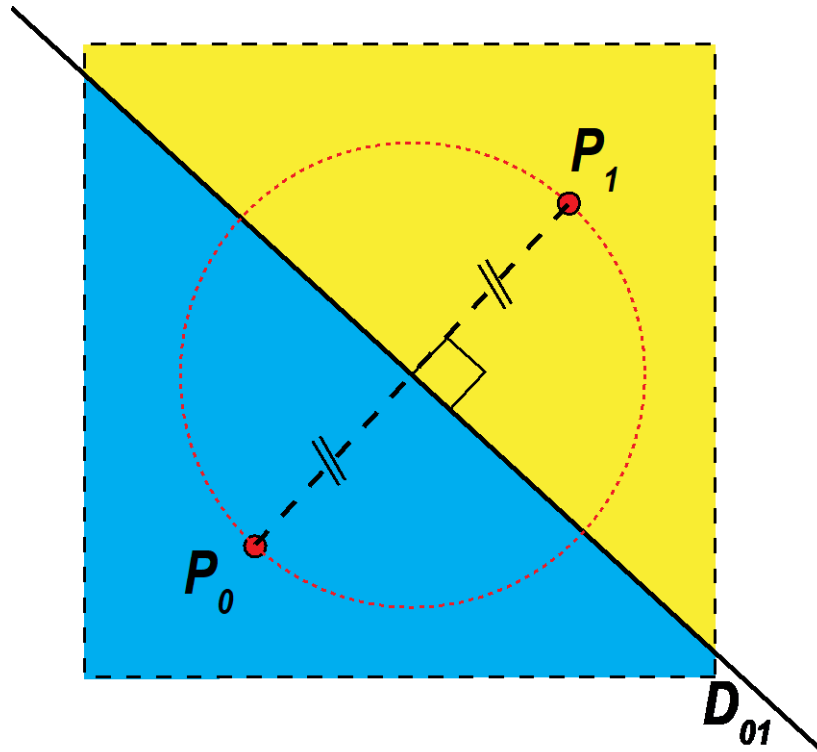
- Given points $P = \{p_i \mid i \in [0 \cdots n - 1]\}$, the Voronoi region of point p_i $V(p_i)$ is the set of points at least as close to p_i as to any other point in P .

$$V(p_i) = \{x \mid |p_i - x| \leq |p_j - x| \forall j \in [0 \cdots i - 1]\}$$

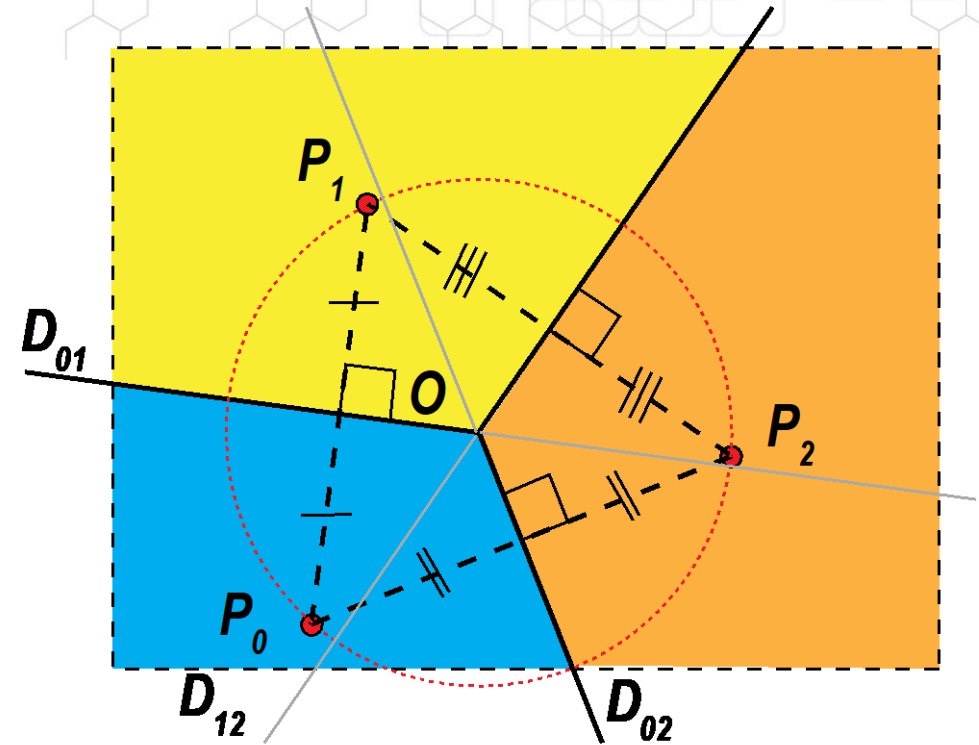
- The Delaunay triangulation is the straight-line dual of the Voronoi Diagram.
 - Circumcenters of Delaunay's triangles are Voronoi diagram vertices.
 - Delaunay's vertices are centers of Voronoi regions.

Voronoi Diagram examples

2 points



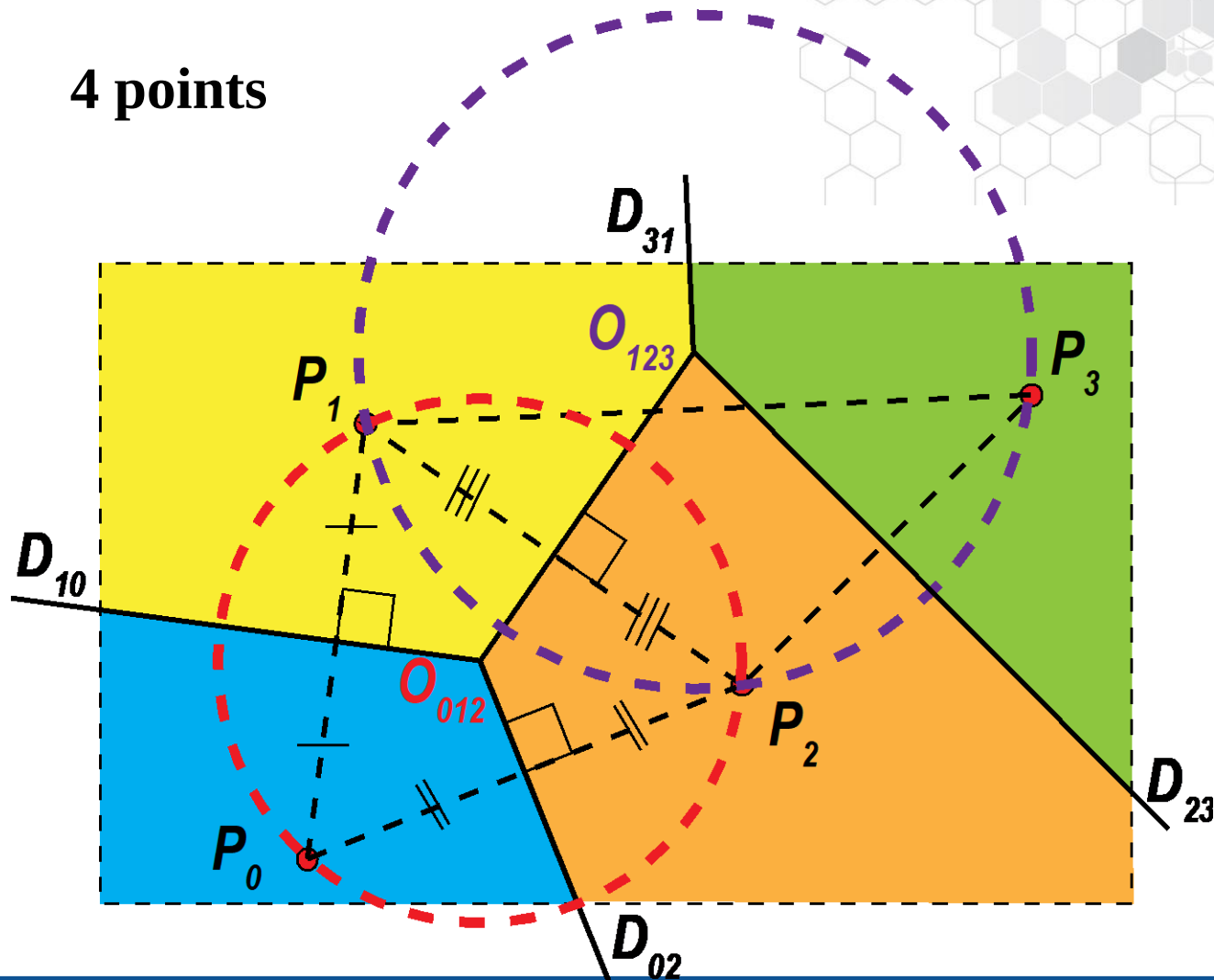
3 points



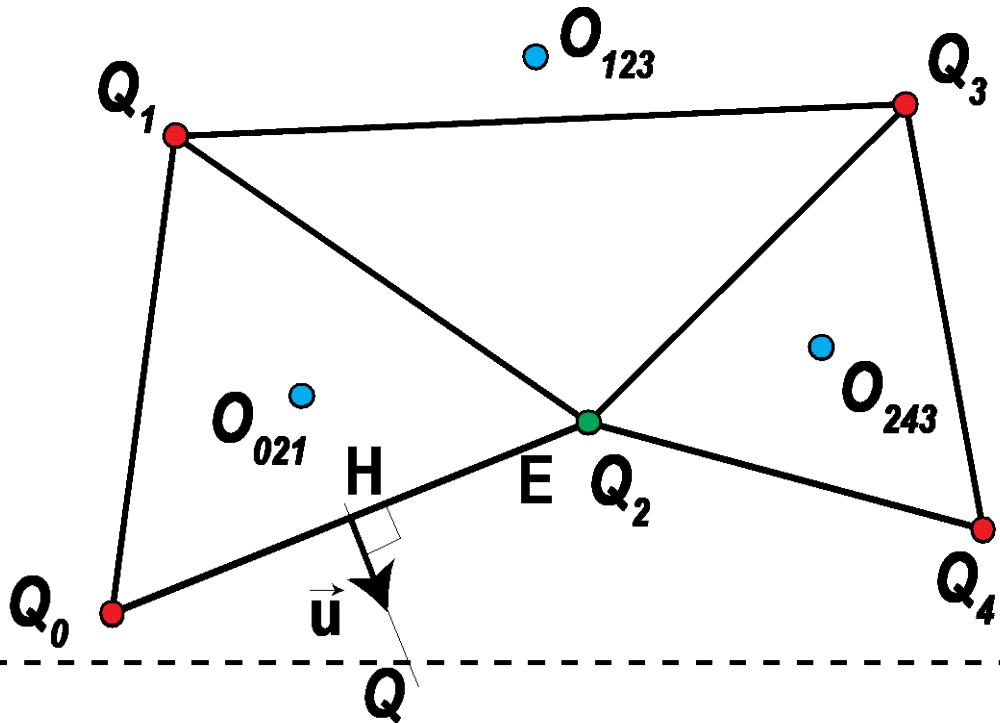
The regions are defined by the perpendicular bisectors of P_iP_j

Voronoi Diagram examples

4 points



Algorithm, from a Delaunay Mesh



Considering Q_2

$T = \{Q_0Q_1Q_2, Q_1Q_3Q_2, Q_3Q_4Q_2\}$

$t = Q_0Q_1Q_2$

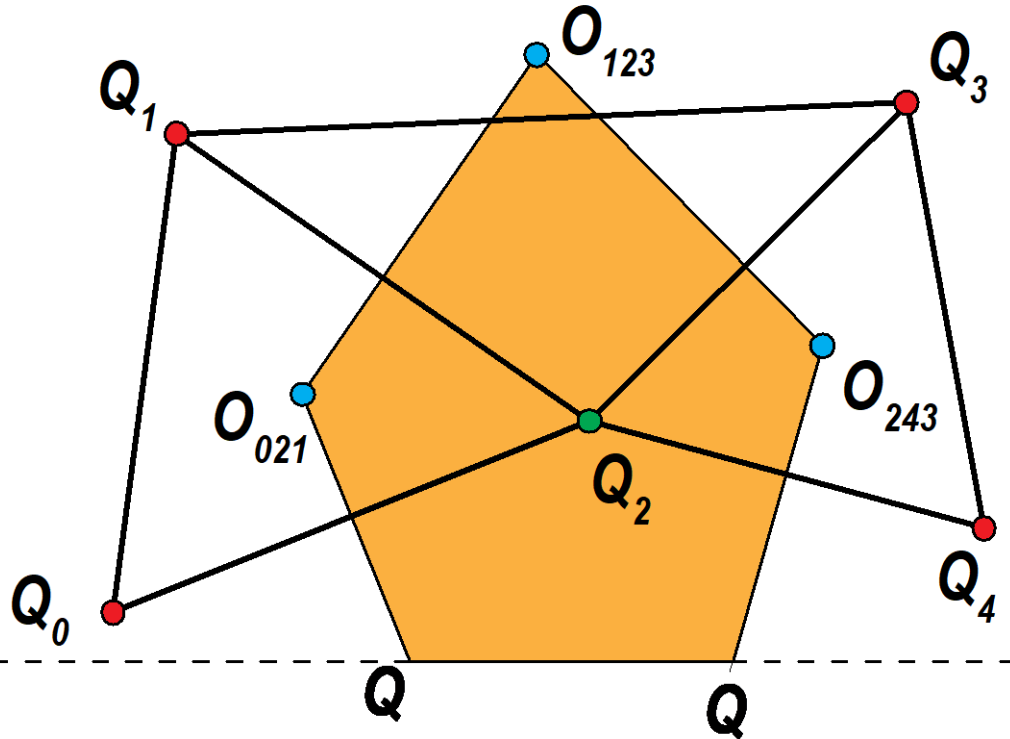
Data: D a Delaunay Mesh

Result: P a list of polygons

```

1 begin
2   foreach vertex  $Q_i$  of  $D$  do
3     Create a new polygon  $P_i$ 
4      $T \leftarrow$  subset of triangles  $t_i(a, b, c)$  of  $D$  where  $Q_i \in \{a, b, c\}$ 
5      $t \leftarrow \text{leftTriangle}(T)$ 
6      $\text{isOpened} \leftarrow (t \text{ exists})$ 
7     if  $\text{isOpened}$  then
8        $E \leftarrow \text{nextEdge}(t, Q_i)$ 
9        $H \leftarrow \text{center of } E$ 
10       $\vec{u} \leftarrow \text{rightOrthogonalVector}(E)$ 
11       $Q \leftarrow \text{IntersectionWithBorders}(H, \vec{u}, 0, 0, \text{width}, \text{height})$ 
12       $\text{addPoint}(Q, P_i)$ 
13    else
14       $t \leftarrow \text{first}(T)$ 
15  end
  
```

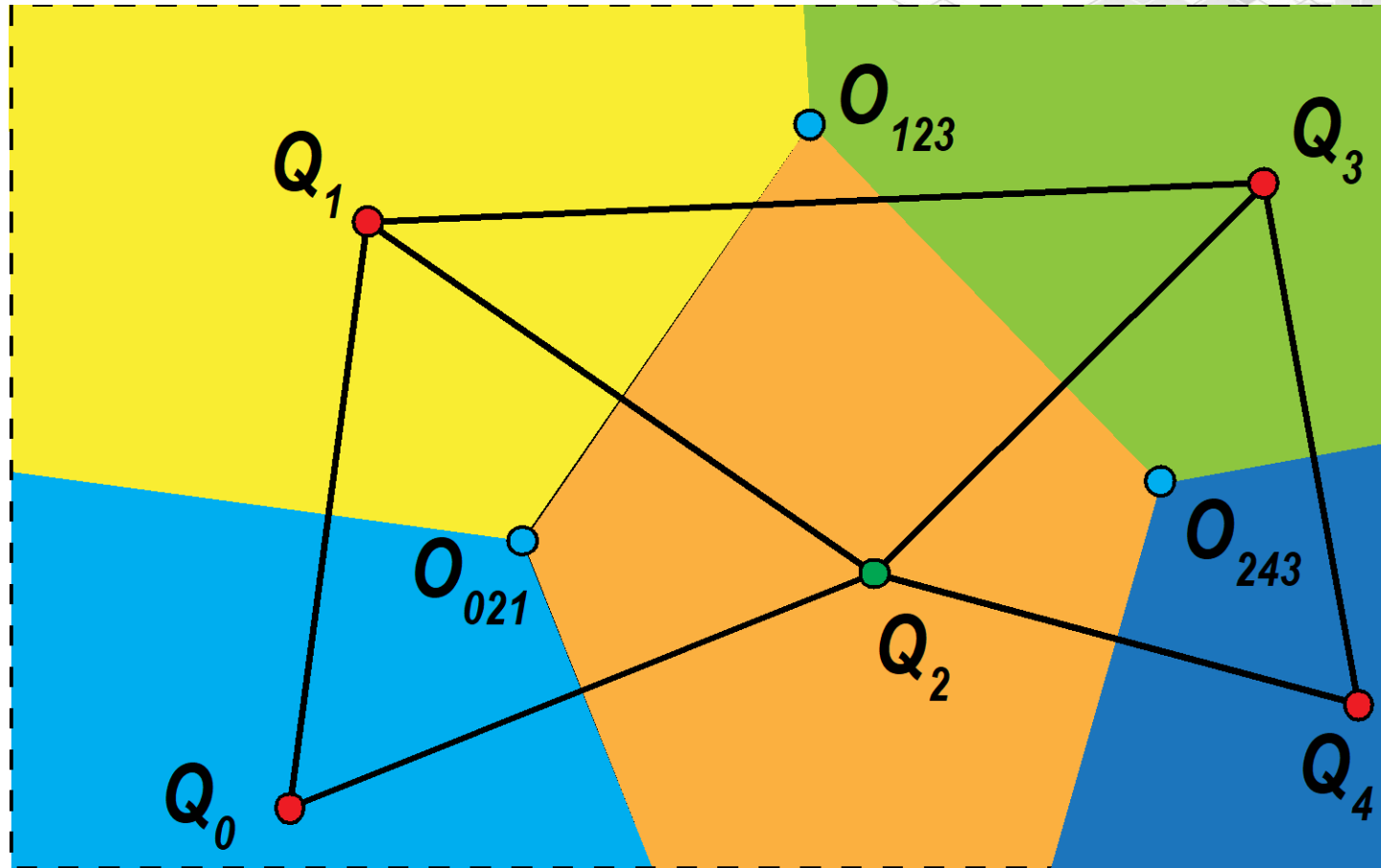

Algorithm, from a Delaunay Mesh



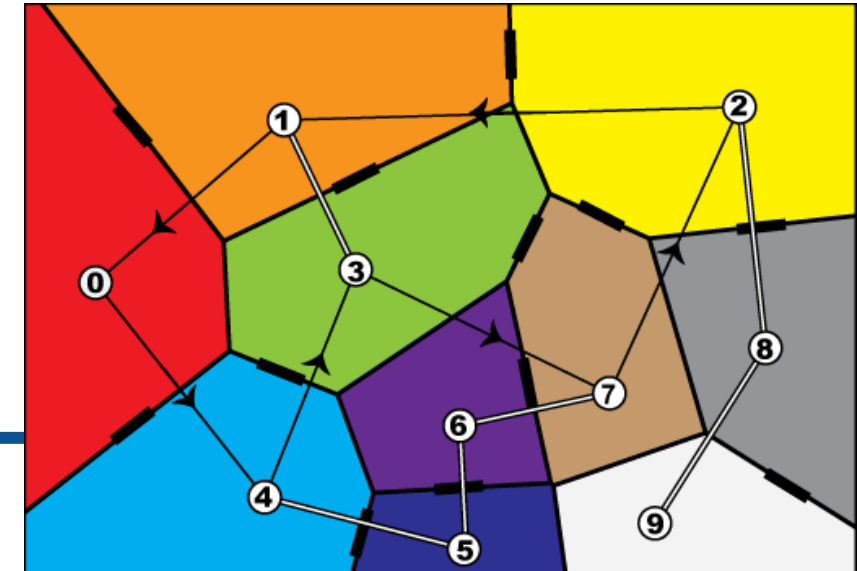
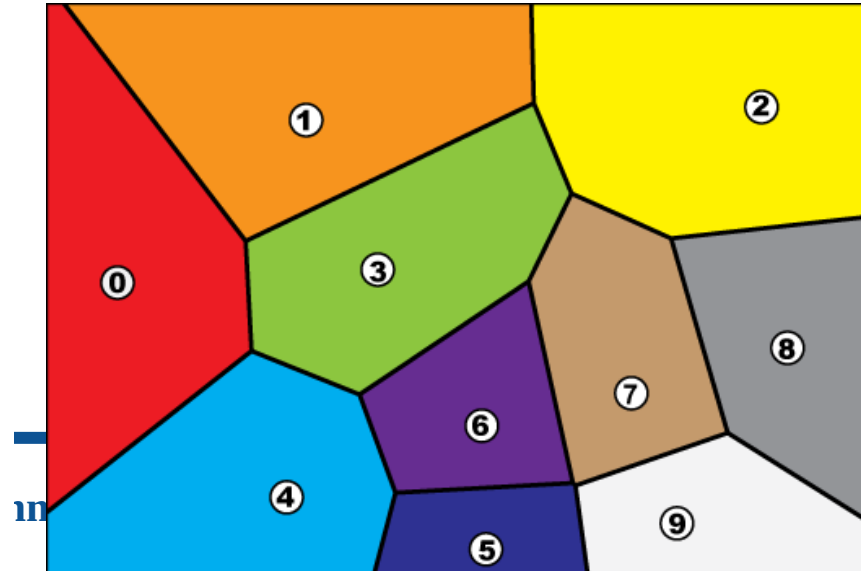
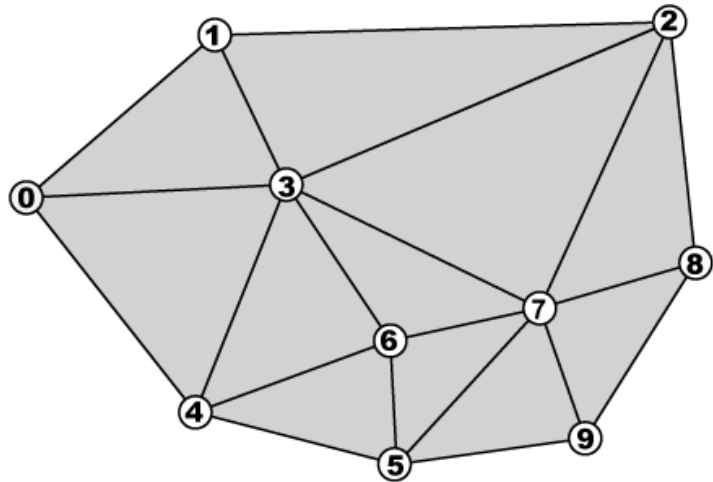
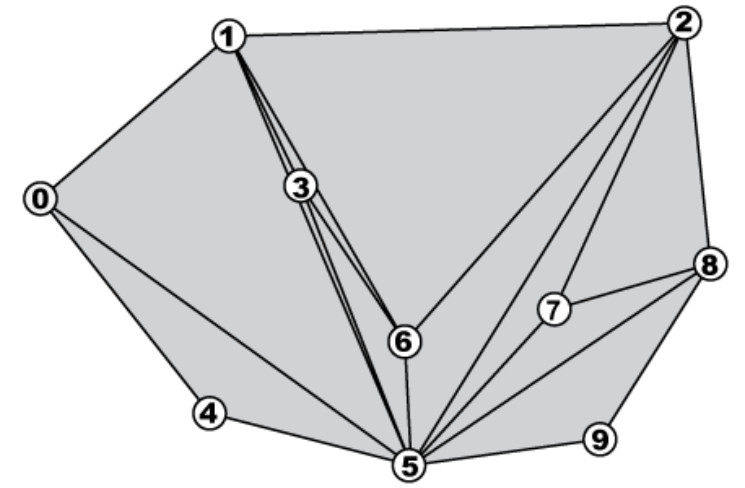
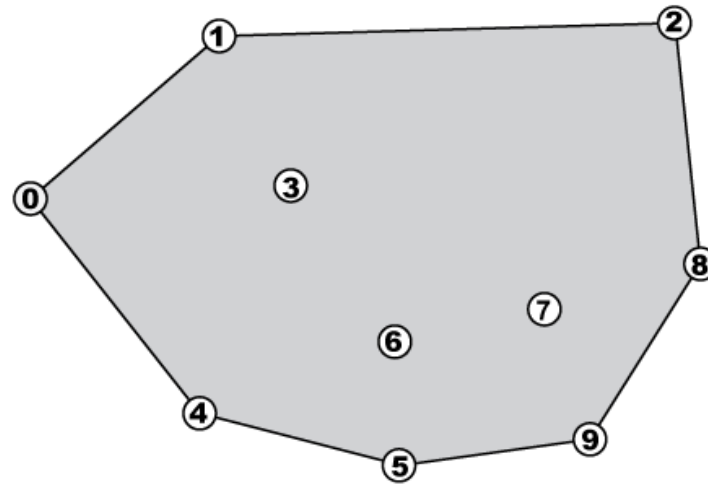
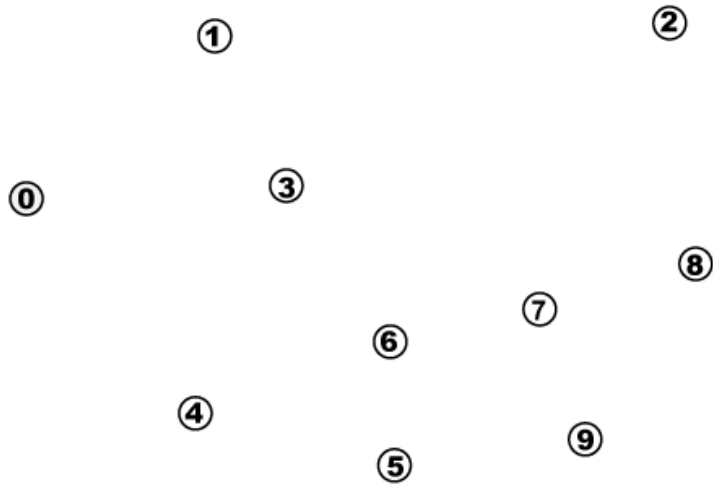
```

16 while size( $T$ ) > 1 do
17     addPoint(circumcenter( $t$ ),  $P_i$ )
18      $t_{prev} \leftarrow t$ 
19      $t \leftarrow \text{rightNeighbor}(t, Q_i)$ 
20     removeTriangle( $t_{prev}, T$ )
21 end
22 addPoint(circumcenter( $t$ ),  $P_i$ )
23 if isOpened then
24      $E \leftarrow \text{prevEdge}(t, Q_i)$ 
25      $H \leftarrow \text{center of } E$ 
26      $\vec{u} \leftarrow \text{rightOrthogonalVector}(E)$ 
27      $Q \leftarrow \text{IntersectionWithBorders}(H, \vec{u}, 0, 0, \text{width}, \text{height})$ 
28     addPoint( $Q, P_i$ )
29 end
30 removeTriangle( $t, T$ )
31 addCornerPoints( $P_i, 0, 0, \text{width}, \text{height}$ )
32 push( $P_i, P$ )
33 end
34 end
    
```

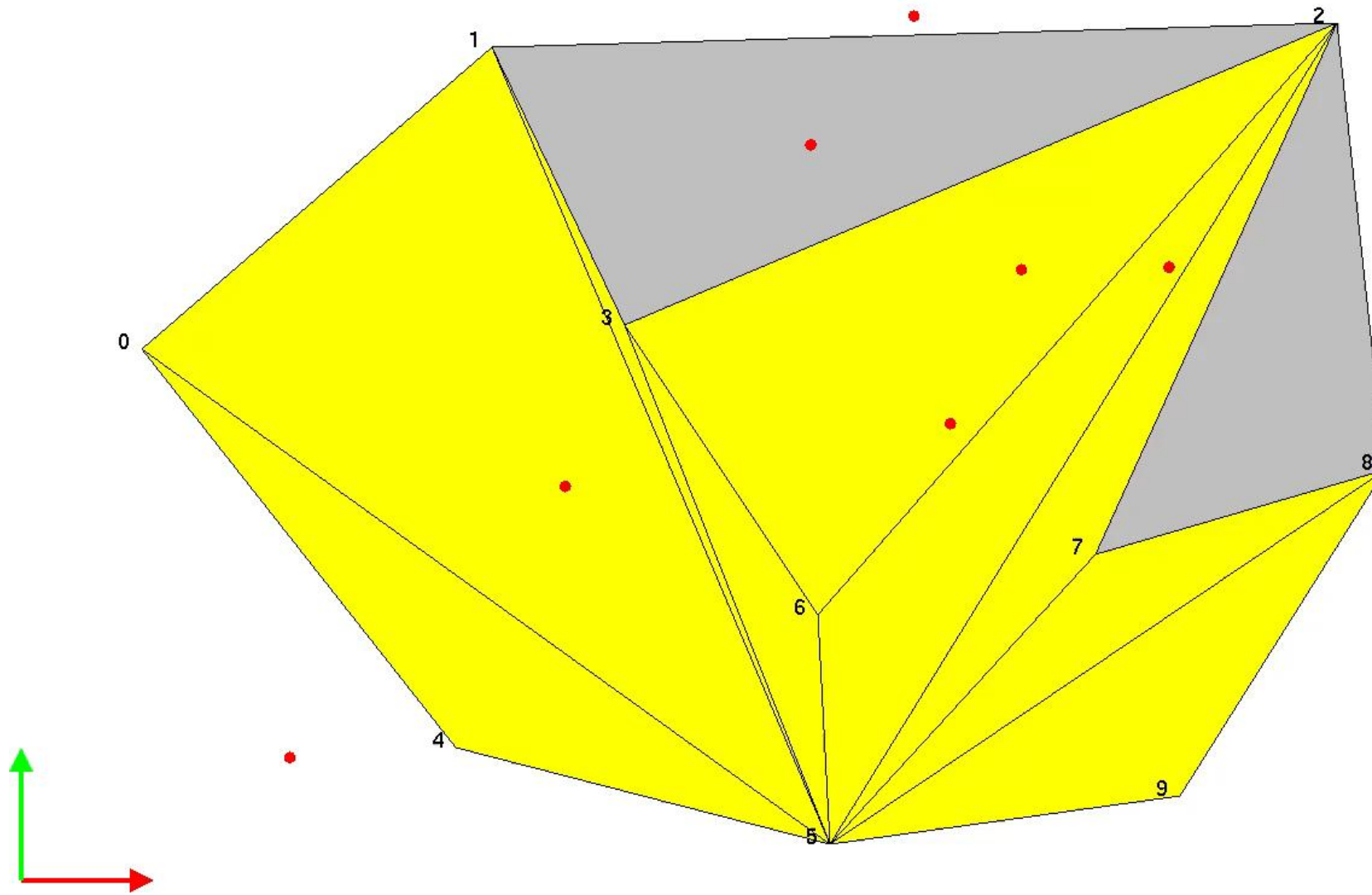
Algorithm, from a Delaunay Mesh



Result on an other configuration



An application: background of a simulation game





3D algorithms

Initiation to image synthesis

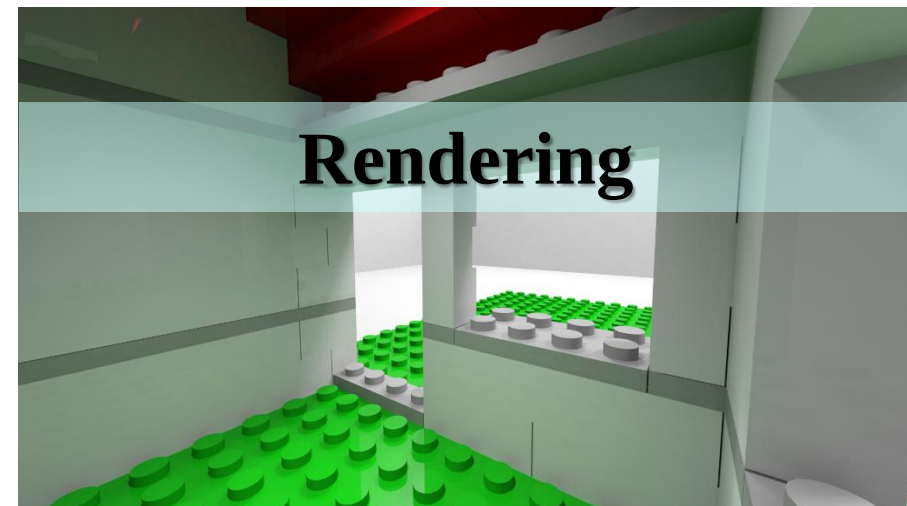
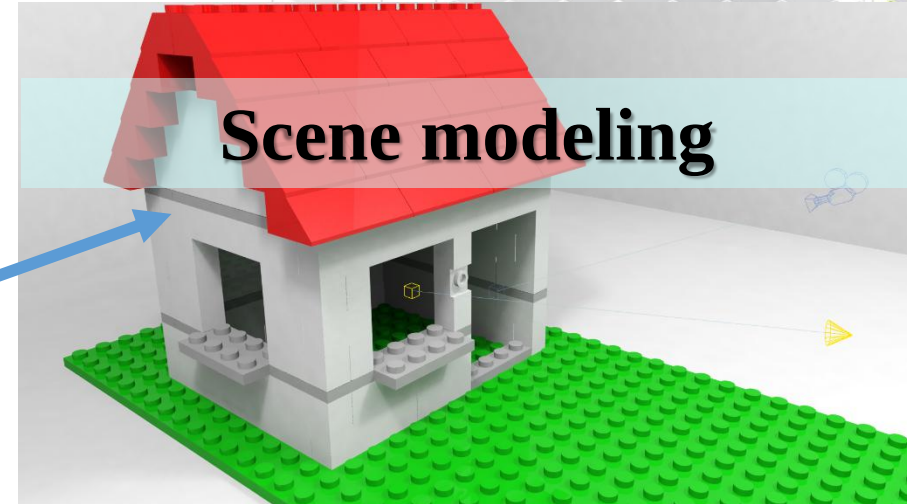
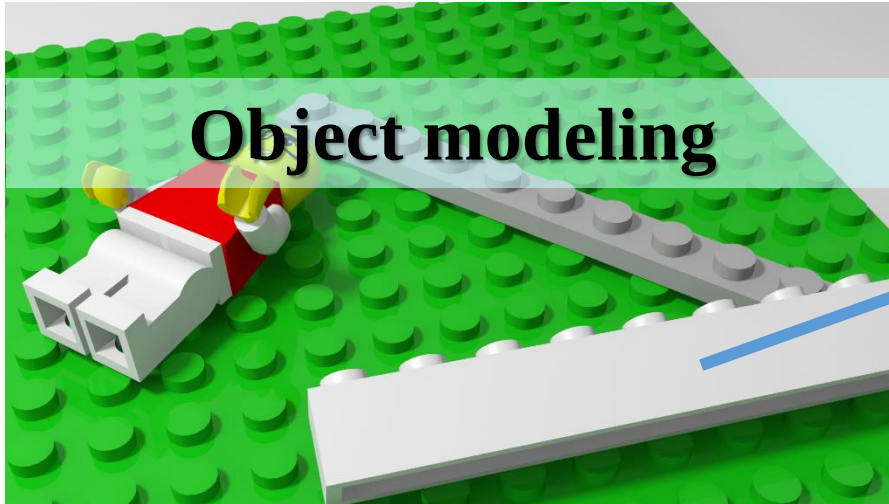
Examples with OpenGL

Image synthesis workflow

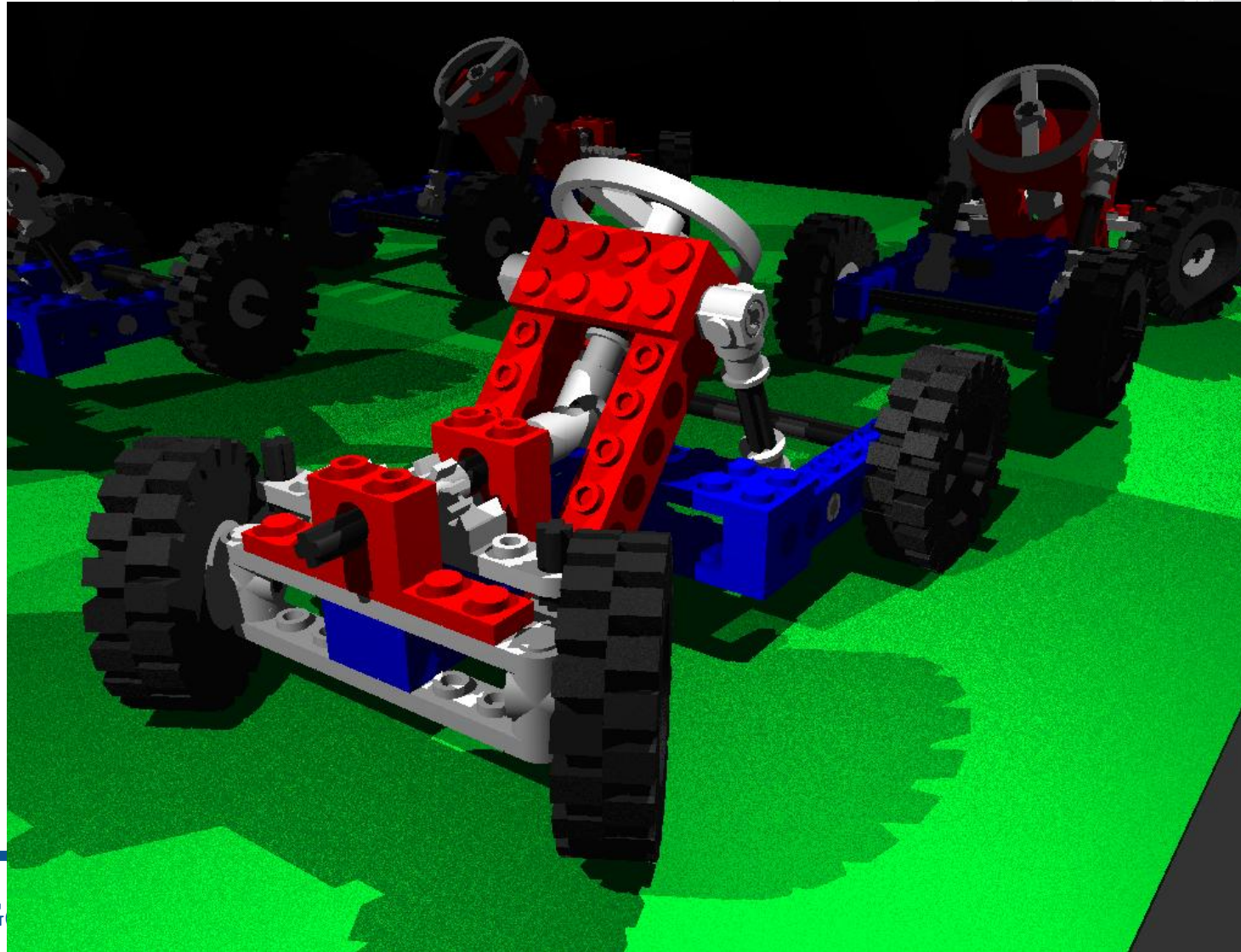


- Goal: create an image
 - Geometric modeling
 - The object
 - The scene
 - Lightning
 - Light $\Leftrightarrow$ materials interactions
 - Rendering algorithms
 - ZBuffer
 - Ray-tracing

From the model to the image



Using Constructive Solid Geometry Tree



Geometry modeling



- Goal
 - Describe the shape of objects (local referential)
 - Place objects in the scene (global referential)
 - Tools participating to the creation of the images
 - Cameras (points of view, direction, view angle)
 - Light sources (position, color, power...)
- Geometrical operations
 - Transform the objects coordinates into the camera coordinate system
 - Project objects on the screen

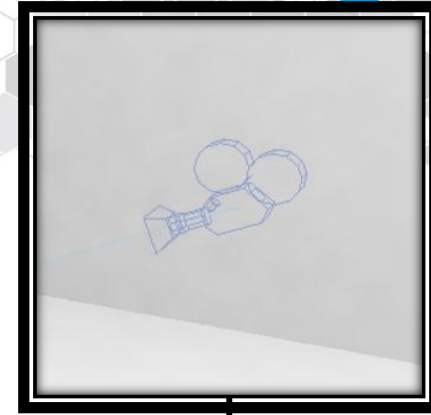
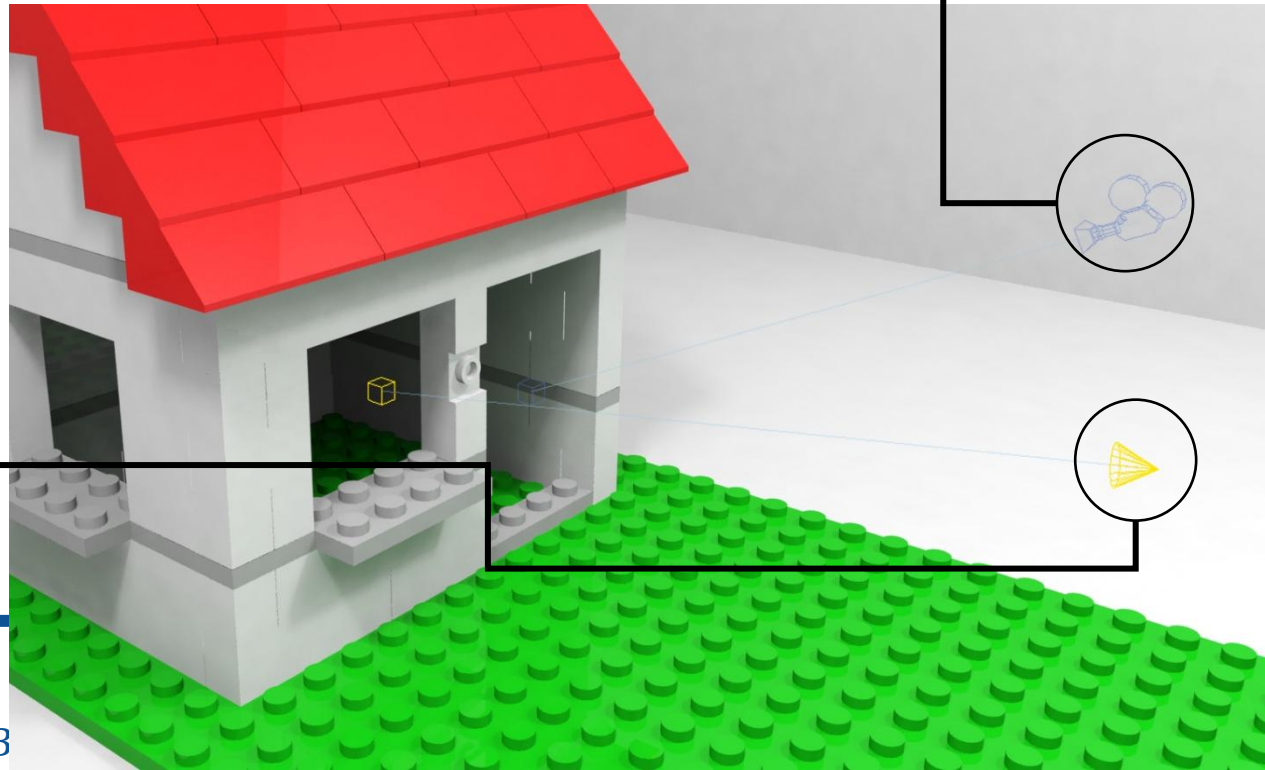
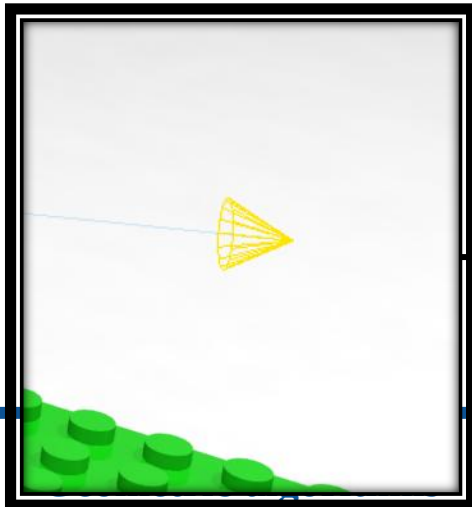
Basic OpenGL 3D functions



- OpenGL drawing tools
 - Basic 3D shapes:
 - GL_TRIANGLE (3N vertices),
 - GL_QUADS (4N vertices),
 - GL_POLYGON (N vertices)
- GLUT Objects
 - glutSolidCube(size)
 - glutSolidSphere(radius,slice,stacks)
 - glutSolidTorus(innerRadius,outerRadius,nsides,rings)
 - glutSolidCone(base,height,slice,stacks)
 - glutSolidTeapot(size)

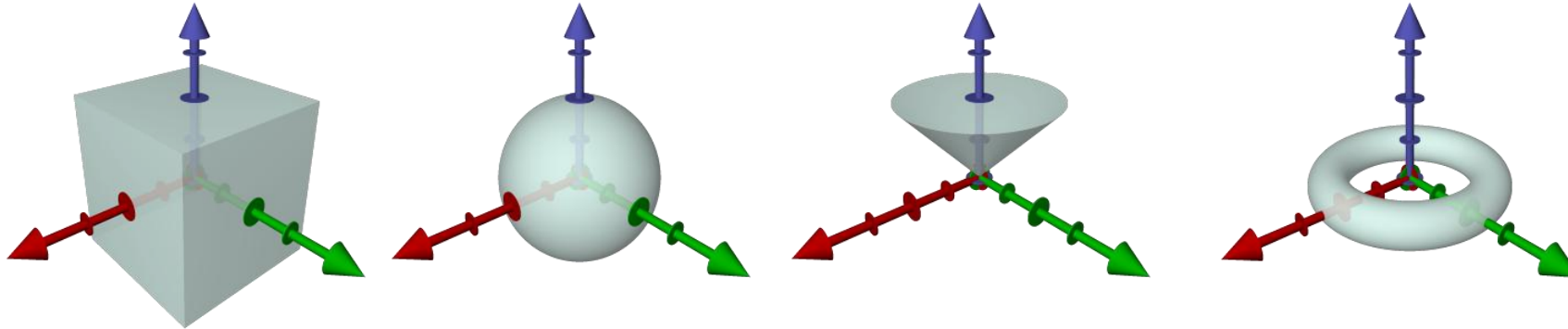
Scene modeling

- Place in the same area
 - 3D Objects
 - Add Cameras
 - Add Light sources
 - ...



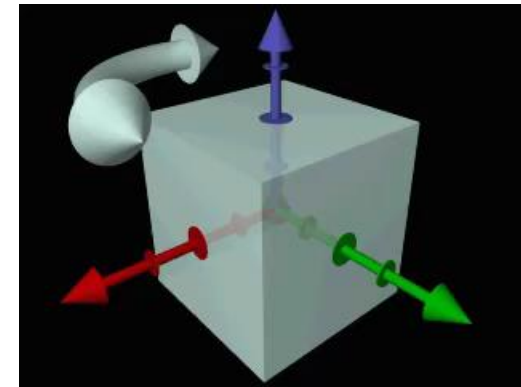
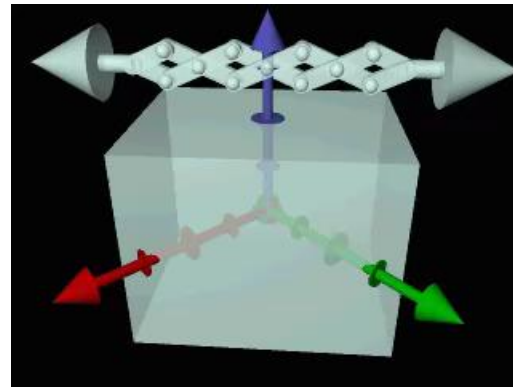
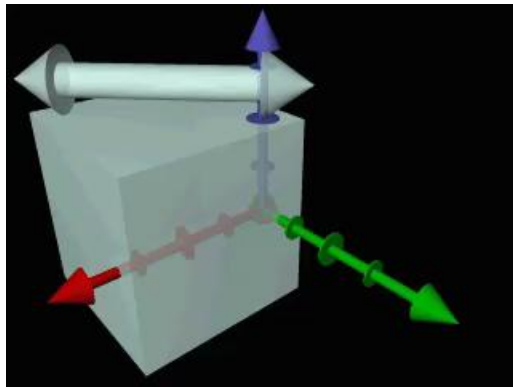
Scene modeling

- Canonical objects
 - Local referential
 - Depending on the shape of the object
 - Unit parameters (radius, height...)
 - No intrinsic modification
 - Apply global geometrical transformations



Scene modeling

- Simple geometrical transformations
 - translation : displacement of objects
 - scale : change the size along axes
 - rotations : rotate object around axes



Simple geometrical transformations

- Translation
 - add a vector $\vec{u}$ to all points of the object

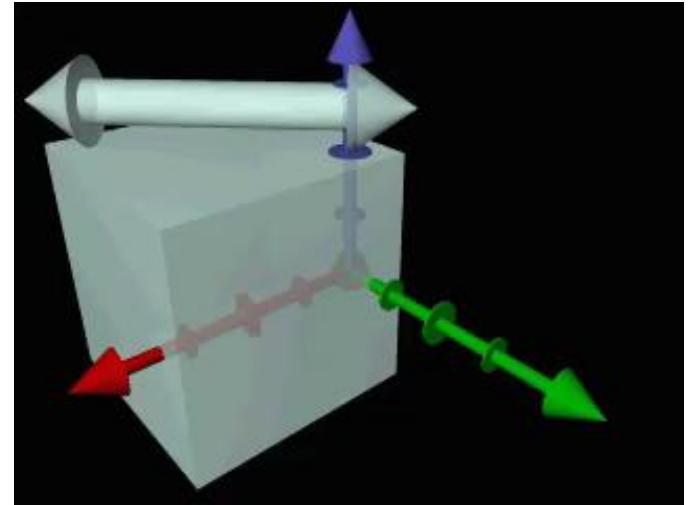
$$t_{\vec{u}}(P) = P + \vec{u}$$

➤ motion of points

- But no effect if applied to a vector

$$t_{\vec{u}}(\vec{v}) = \vec{v}$$

➤ dimensions and angles unchanged

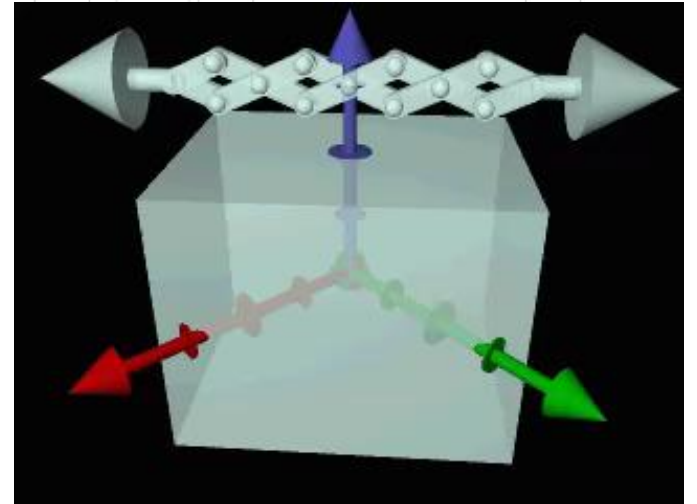


Simple geometrical transformations

- Scale
 - multiply point or vector components by coefficients

$$h_{(a,b)} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax \\ by \end{pmatrix}$$

- Move points relatively to the origin,
- Scale the lengths,
- Change angles if $a \neq b$
- Do not affect the origin

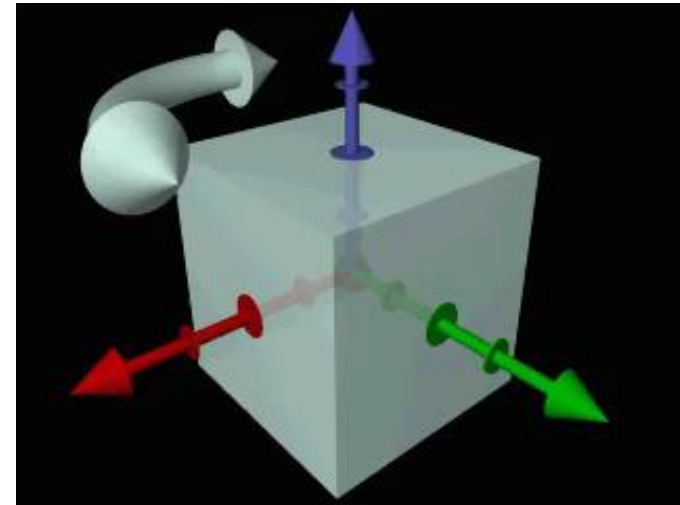
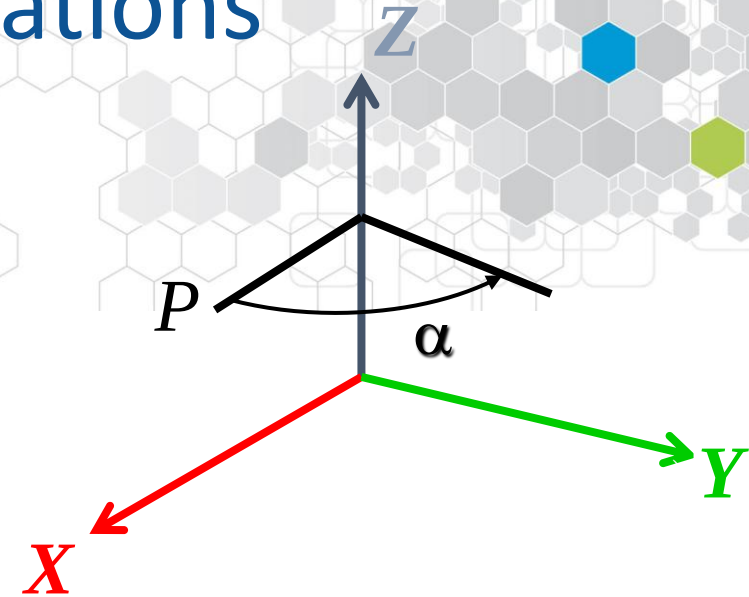


Simple geometrical transformations

- Rotation
 - rotation around an axis of a given angle.

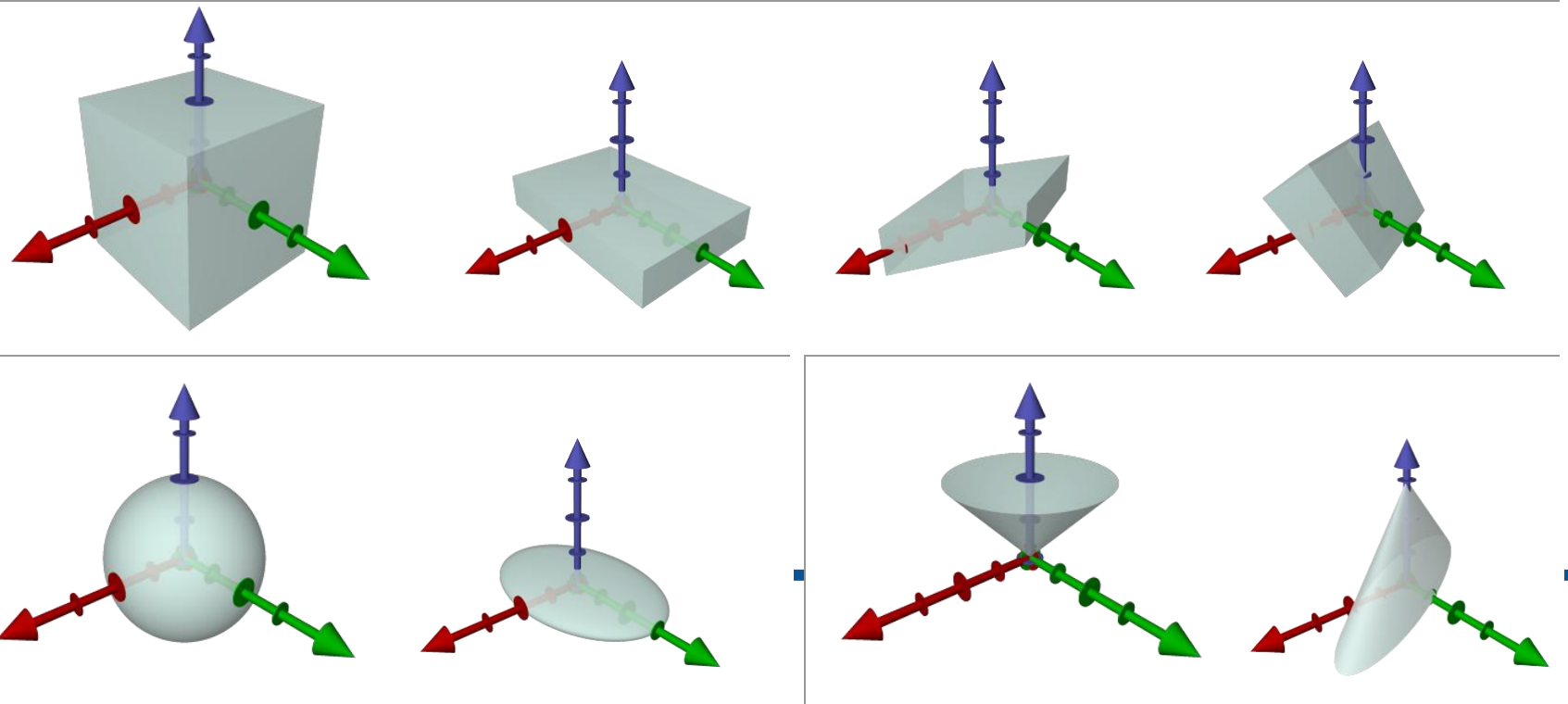
➤ Do not affect axis points

$$r_{Z(\alpha)} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \cos(\alpha)x - \sin(\alpha)y \\ \sin(\alpha)x + \cos(\alpha)y \\ z \end{pmatrix}$$



Canonical object transformations

- We get a family of objects
 - sphere -> ellipsoids
 - cube -> parallelepipeds
 - cylinder -> elliptic base cylinders



Geometrical transformations

- Transformation in $\mathbb{R}^3$

$$r_{Z(\alpha)}(P) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} P$$

$$h_{(a,b,c)}(P) = \begin{pmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{pmatrix} P$$

$$t_{\vec{u}}(P) = P + \vec{u}$$

Common expression: using homogeneous matrices

Homogeneous matrices

- Problem: write a matrix that applies a linear combination in $\mathbb{R}^3$

$$M \times P \Leftrightarrow A \times P + B$$

- Solution: using an homogenous expression of dimension 4

$$\begin{pmatrix} \begin{pmatrix} A \end{pmatrix} \\ 0 \quad 0 \quad 0 \end{pmatrix} \begin{pmatrix} B \\ 1 \end{pmatrix} \times \begin{pmatrix} P \\ 1 \end{pmatrix} = A \times P + B$$

Homogeneous matrices

- Translation homogeneous matrices

- Applied to a point P

$$T_{\vec{u}}P = \begin{pmatrix} 1 & 0 & 0 & \vec{u}_x \\ 0 & 1 & 0 & \vec{u}_y \\ 0 & 0 & 1 & \vec{u}_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} P \\ 1 \end{pmatrix} = P + \vec{u}$$

- Applied to a vector V

$$T_{\vec{u}}\vec{V} = \begin{pmatrix} 1 & 0 & 0 & \vec{u}_x \\ 0 & 1 & 0 & \vec{u}_y \\ 0 & 0 & 1 & \vec{u}_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} \vec{V} \\ 0 \end{pmatrix} = \vec{V}$$

- The expression of a **point** and a **vector** are different using homogeneous coordinates!

Homogeneous matrices

- Rotation around Z axis homogeneous matrix

$$R_{Z(\alpha)} = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Scale homogeneous matrix

$$H_{(a,b,c)} = \begin{pmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Composing transformations

- If we apply several transformations
 - Can be coded in a simple matrix,
 - Is the product of transformation matrices,
 - Be careful with the order of transformations!

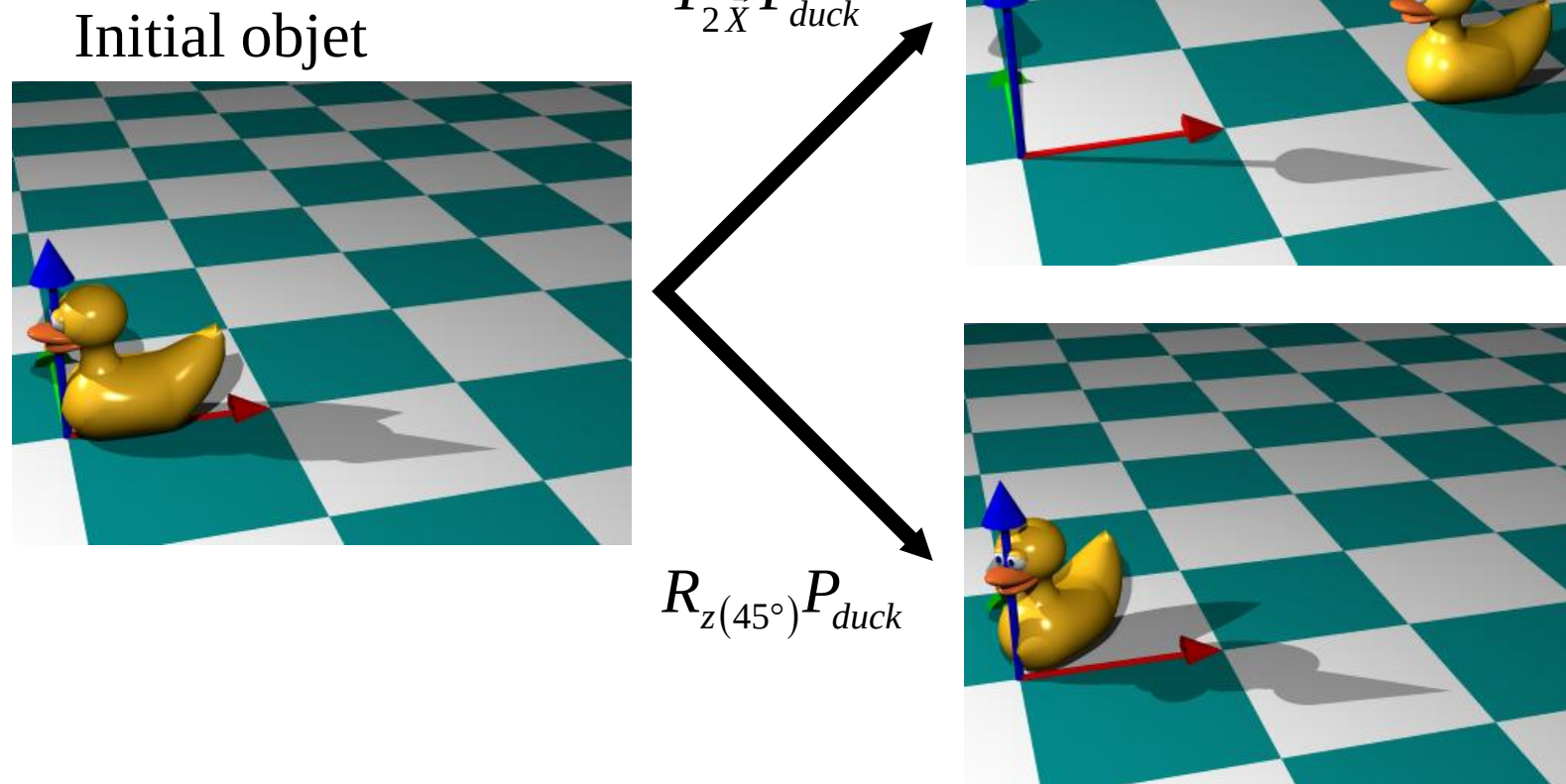
Rotation of angle α around Z followed by a translation of vector $\vec{u}$

$$t_{\vec{u}}(r_{Z(\alpha)}(\text{Objet})) \Leftrightarrow T_{\vec{u}}(R_{z(\alpha)}P_{\text{objet}}) = (T_{\vec{u}} \times R_{z(\alpha)})P_{\text{objet}}$$

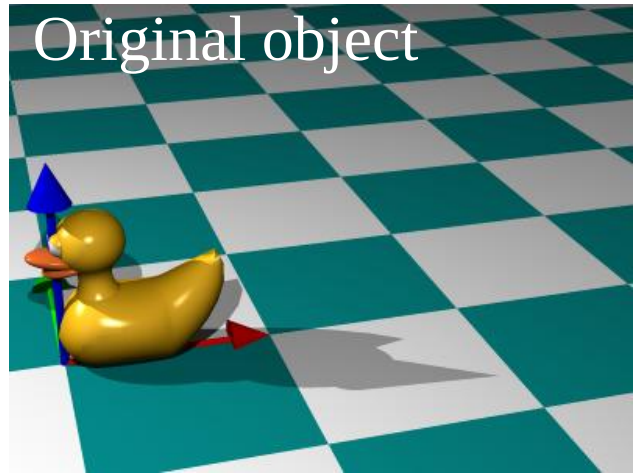
⚡ Be careful with the order

$$(T_{\vec{u}} \times R_{z(\alpha)})P_{\text{objet}} \neq (R_{z(\alpha)} \times T_{\vec{u}})P_{\text{objet}}$$

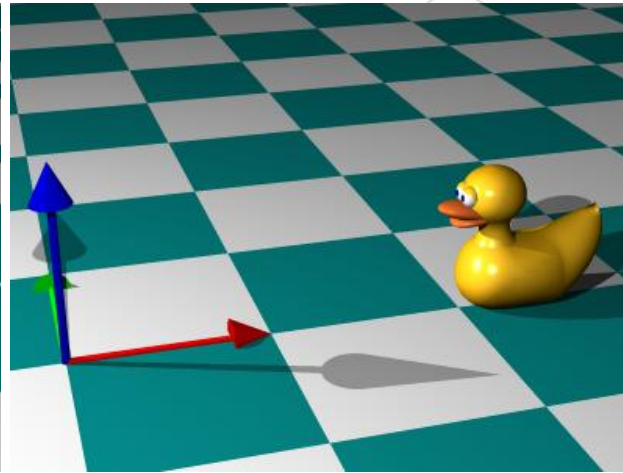
Examples of compositing



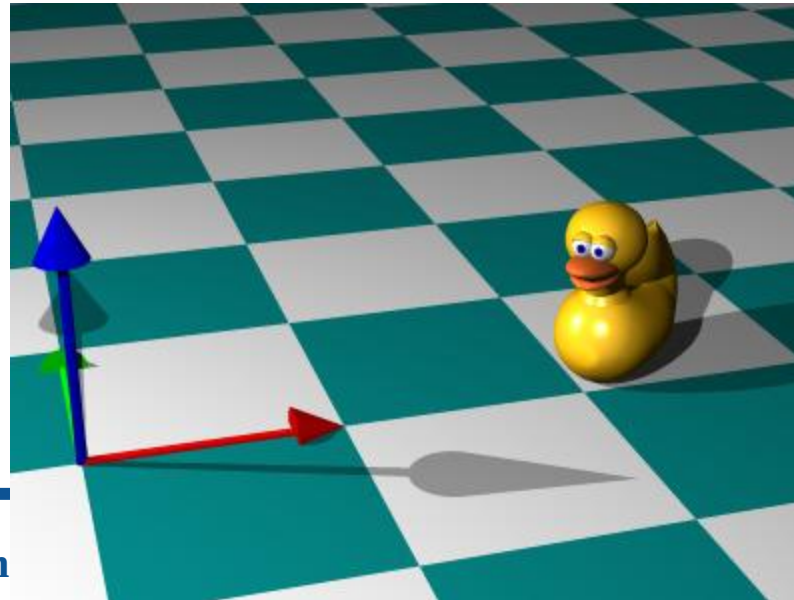
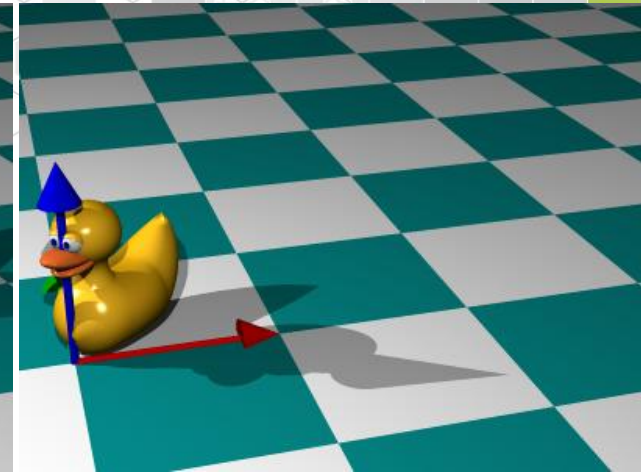
First order



$$T_{2\bar{X}} P_{duck}$$

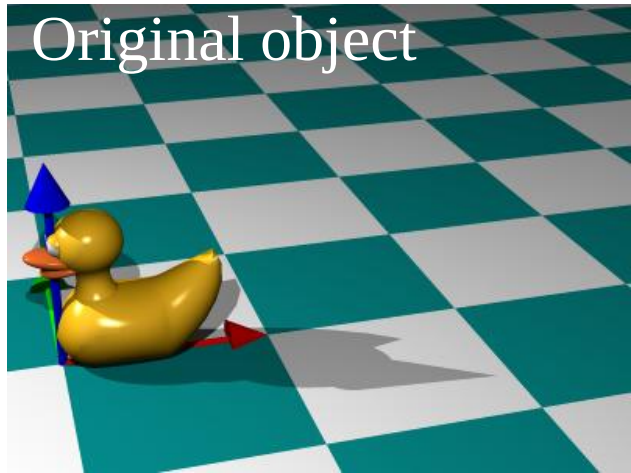


$$R_{z(45^\circ)} P_{duck}$$

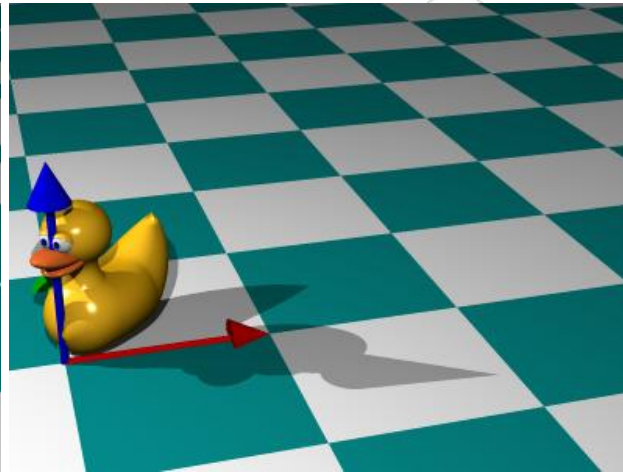


$$(T_{2\bar{X}} \times R_{z(45^\circ)}) P_{duck}$$

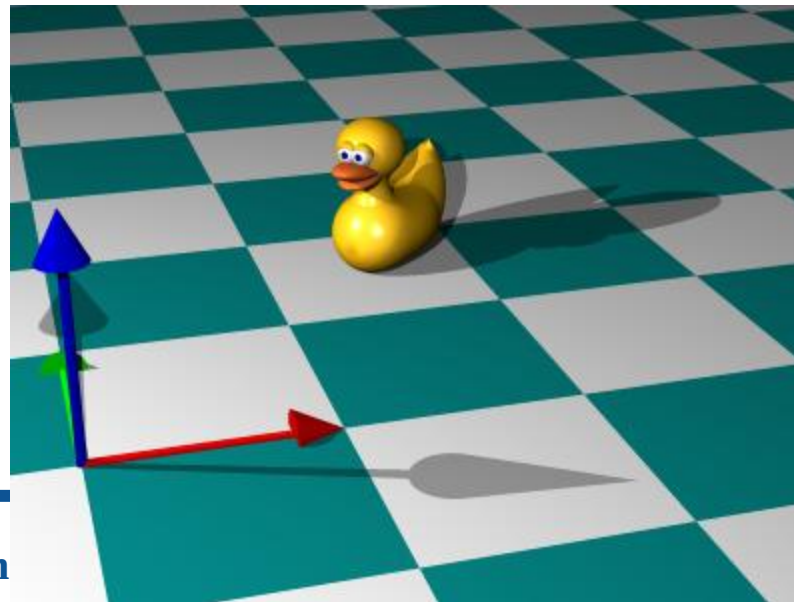
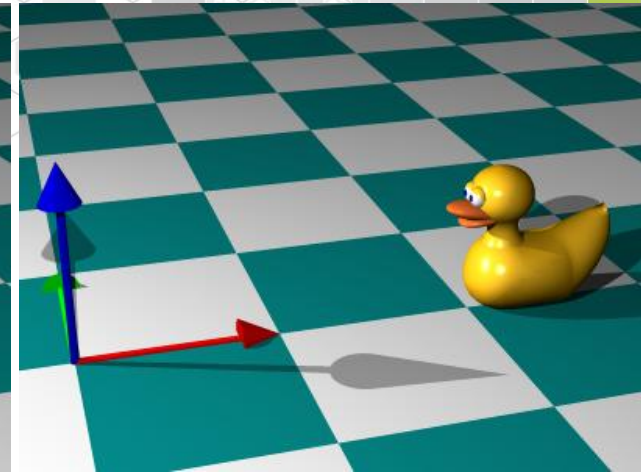
Second order



$$R_{z(45^\circ)} P_{duck}$$



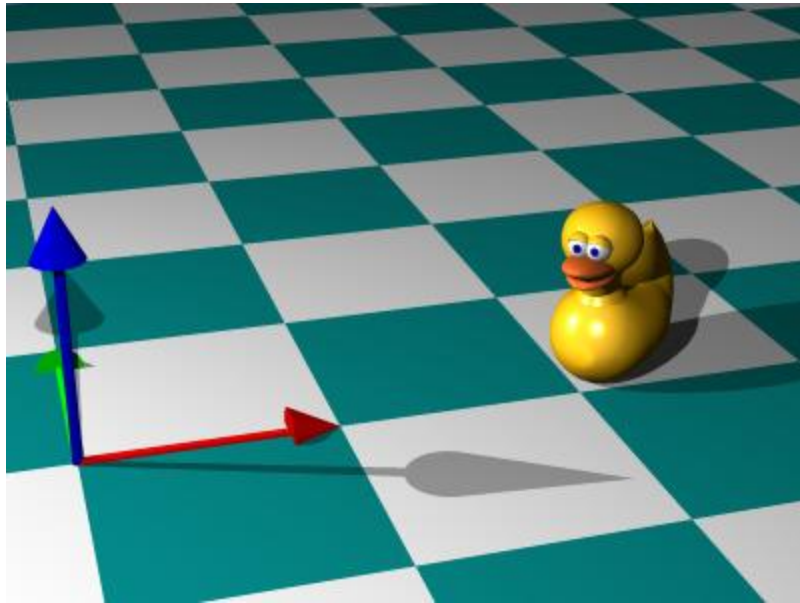
$$T_{2\vec{X}} P_{duck}$$



$$(R_{z(45^\circ)} \times T_{2\vec{X}}) P_{duck}$$

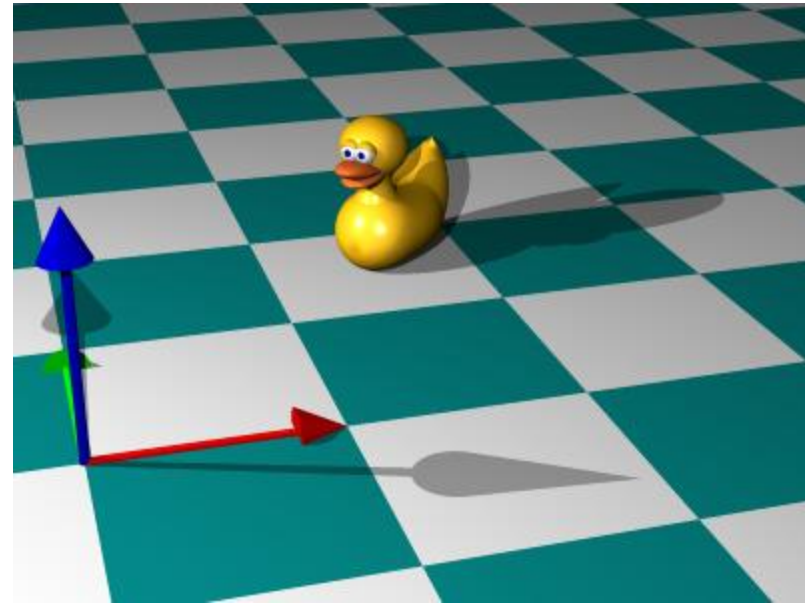
Order of combination

$$(T_{2\vec{X}} \times R_{z(45^\circ)})P_{duck}$$



$\neq$

$$(R_{z(45^\circ)} \times T_{2\vec{X}})P_{duck}$$



OpenGL scene modelisation



- Uses Homogeneous matrices
 - `GL_PROJECTION_MATRIX`: for the camera and projection into the scree,
 - `GL_MODELVIEW_MATRIX`: for the scene
 - `glInitMatrix()`: set the current matrix to Identity
 - `glTranslatef(x,y,z)`: multiply the current matrix with the translation matrix
 - `glRotatef(angle,x,y,z)`: multiply the current matrix with the rotation matrix
 - `glScalef(x,y,z)`: multiply the current matrix with the scale matrix
 - `glPushMatrix()`: store the current matrix into a stack
 - `glPopMatrix()`: set the matrix at the top at the stack to the current matrix and pop it.

Example of usage

```
glPushMatrix();  
glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, blue);  
glutSolidCylinder(0.02, 2.0, 20, 5);  
glPushMatrix();  
glRotatef(-90.0, 1, 0, 0);  
glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, green);  
glutSolidCylinder(0.02, 2.0, 20, 5);  
glPopMatrix();  
glRotatef(90.0, 0, 1, 0);  
glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, red);  
glutSolidCylinder(0.02, 2.0, 20, 5);  
glPopMatrix();
```

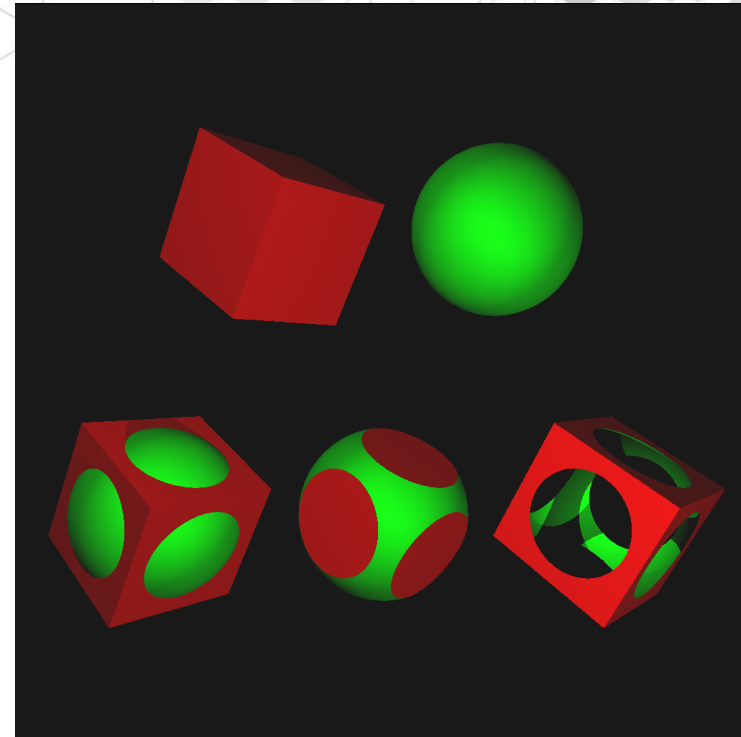
Scene modeling



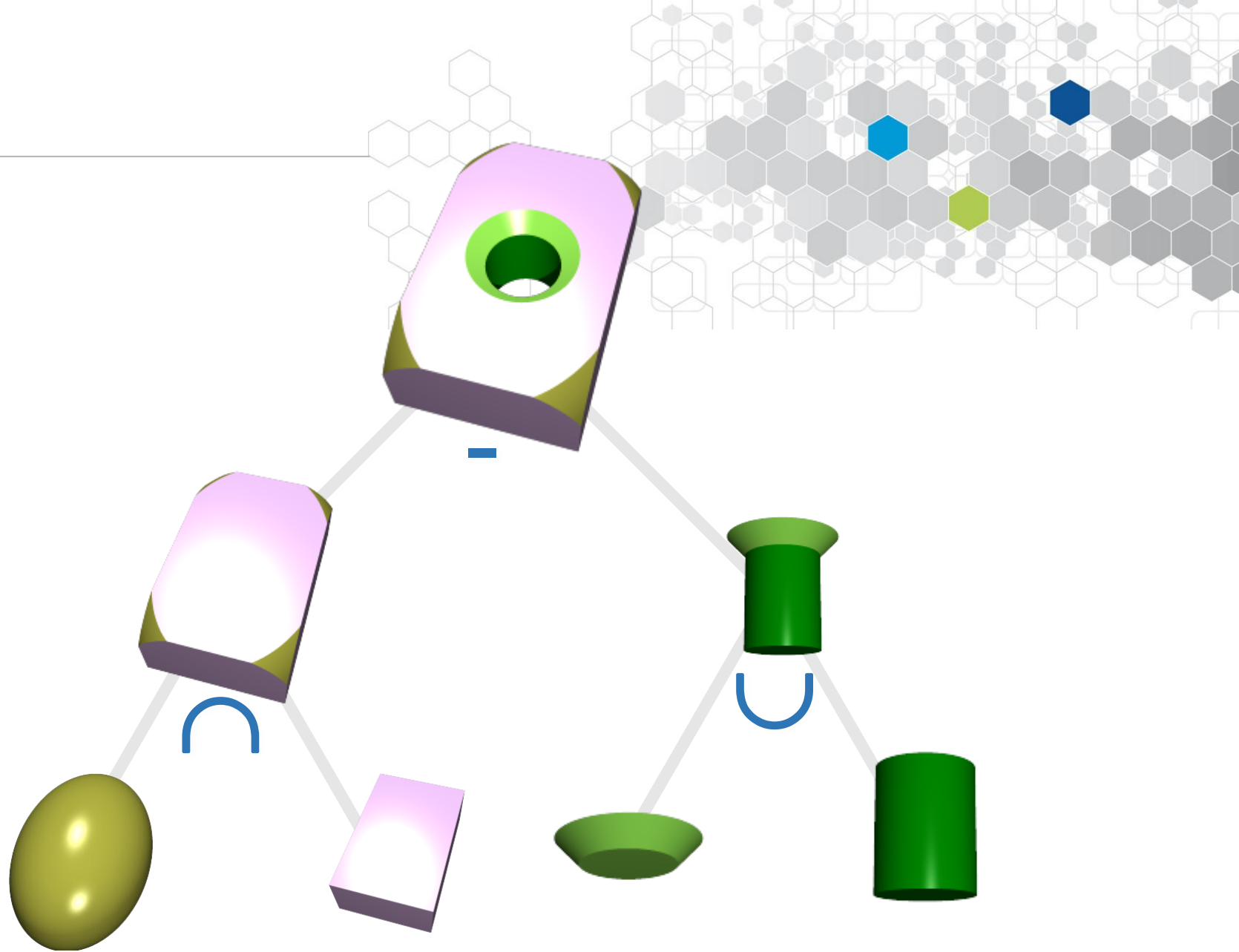
- CSG Model : Constructive Solid Geometry Tree
 - Describe a scene by a tree
- CSG Tree
 - Leaves:
 - canonical objects
 - Internal nodes:
 - Sub-trees compositing operators
 - Union
 - Intersection
 - Difference
 - Geometrical operators
 - Translation
 - Rotation
 - Scale

Compositing operators

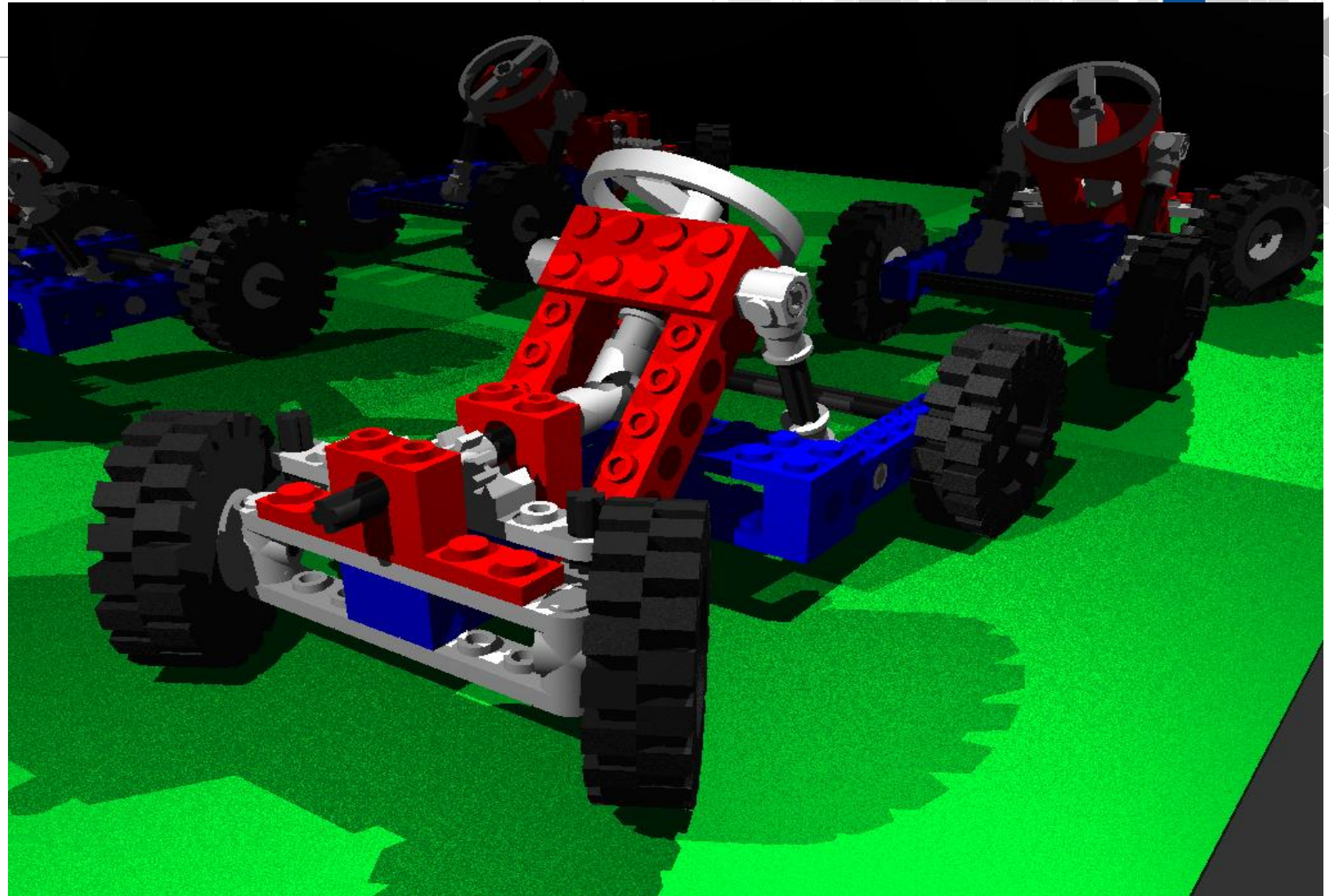
- Union
 - volume filled by at least one of the sub-trees.
- Intersection
 - Common volume of sub-trees.
- Différence
 - volume of the first sub-tree but not filled by the others.



CSG Tree



Full scene

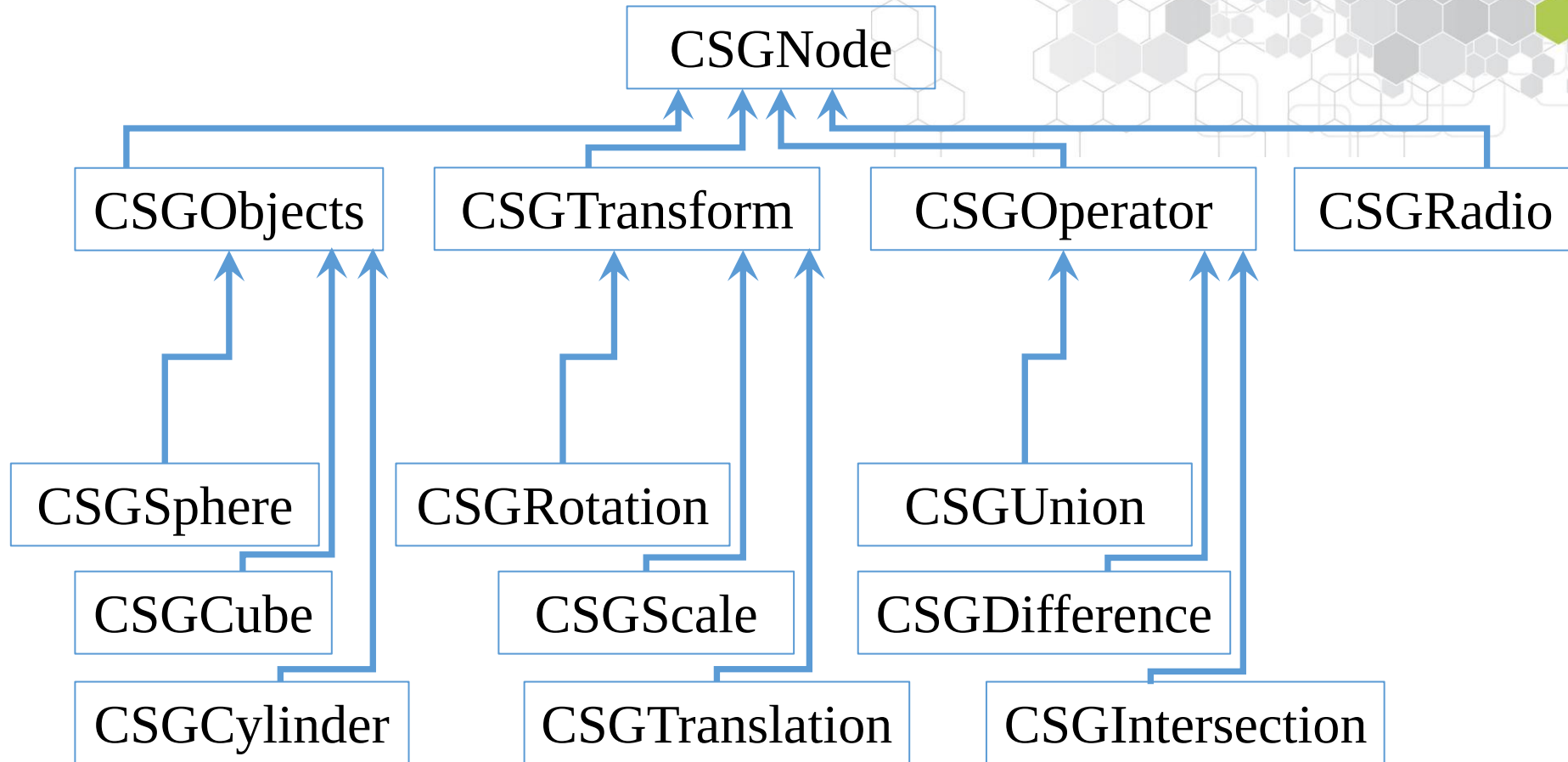


Programming CSG trees



- Definition of a set of classes
 - Inheritance and polymorphism
 - We define a main class CSGNode
 - We define derived classes for each kind of node (CSGUnion, CSGSphere,...)
 - Attributs:
 - CSGNode : int id
 - CSGTransform : Matrix mat,mat_1
 - CSGObject : string nom
 - Methods
 - glDraw() to draw each node (internal node too!)
 - Picking() to calculate the intersections between a ray and the scene

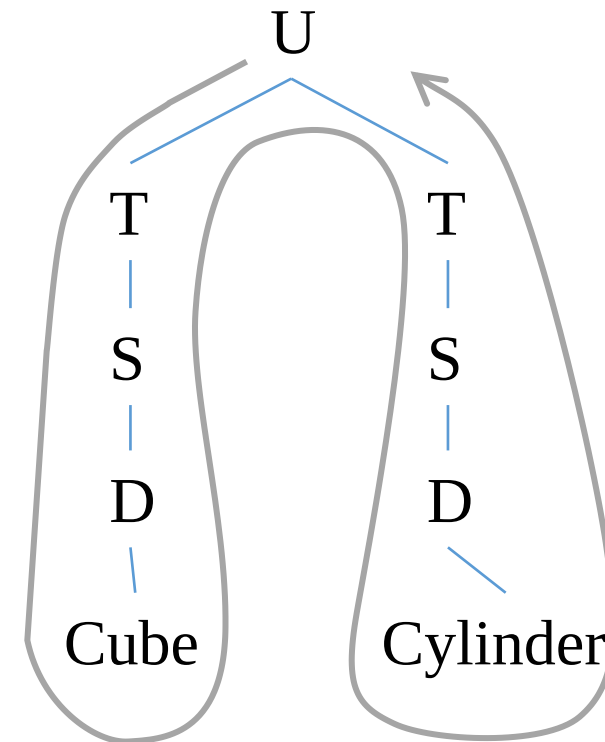
Classes



CSG tree / associated word

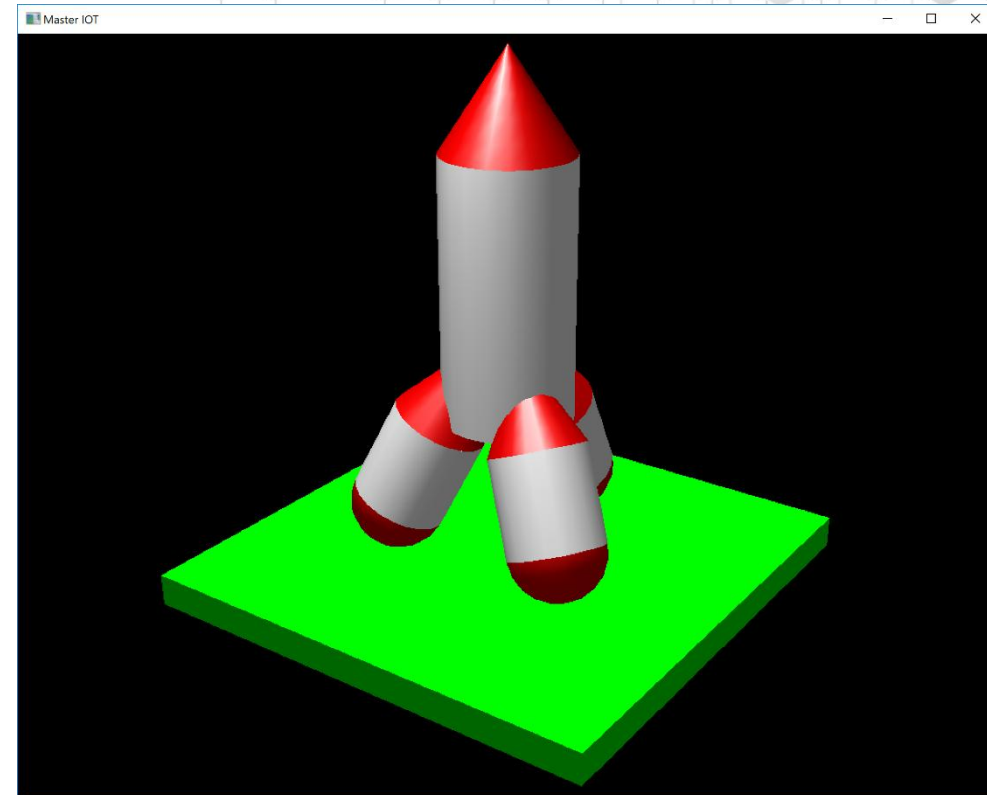
- We can write an instance of a CSG tree by a word
- Prefix depth first search algorithm
 - keyword + parameters

U 2	<i>union + 2 children</i>
T -1,0,-2.1	<i>translation + vecteur</i>
S 5,2,0.1	<i>scale + coefficients</i>
A&D 1.0,0.0,0.5	<i>diffusion + color</i>
CUB sol	<i>Cube + name</i>
T 1,0,-2	<i>translation + vector</i>
S 0.5,0.5,4	<i>scale + coefficients</i>
A&D 0.0,1.0,0.5	<i>diffusion + color</i>
CYL tube	<i>Cylinder + name</i>



CSG tree / associated word

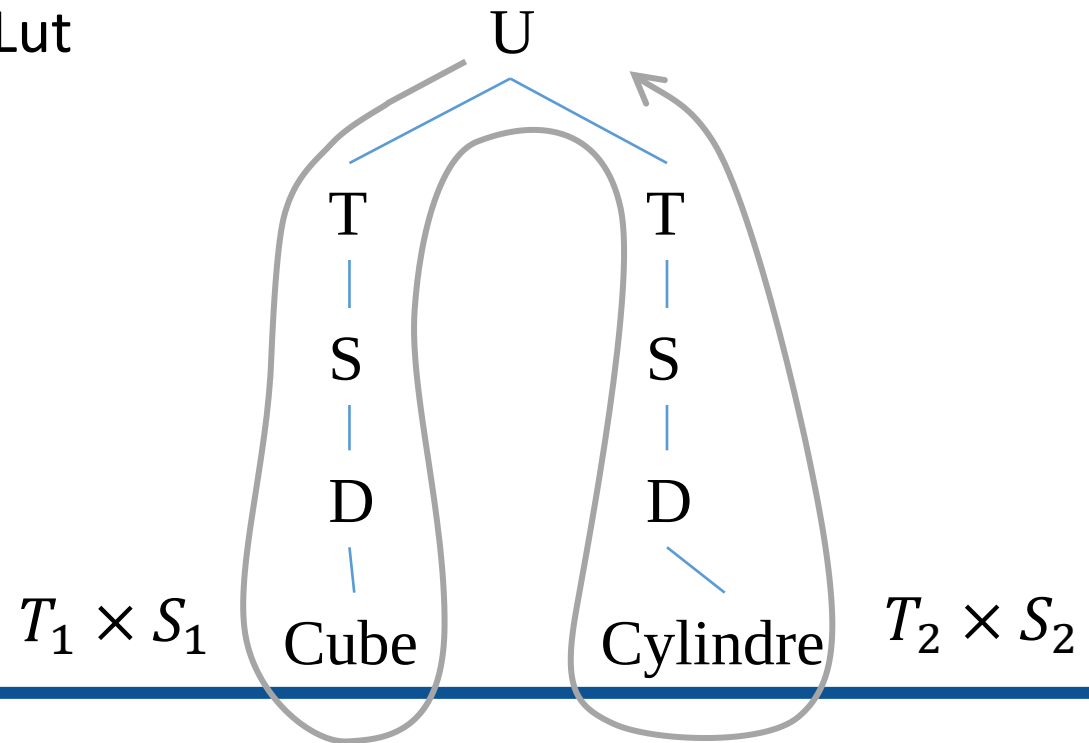
U 2	RZ 120
A&D 0.0,1.0,0.0	T 2.034,0,-0.25
SPC 1.0,1.0,1.0 100.0	RY -26.565
T 0,0,-4.93	U 3
S 15,15,1	S 1.5,1.5,3
CUB sol	A&D 0.8,0.0,0.0
A&D 1.0,1.0,1.0	CON pied02
SPC 1.0,1.0,1.0 20.0	T 0,0,-3
U 5	S 1.5,1.5,3
S 2,2,8	A&D 1.0,1.0,1.0
A&D 1.0,1.0,1.0	CYL tubePied02
CYL corps	T 0,0,-3
T 0,0,8	S 1.5,1.5,1.5
S 2,2,3	A&D 0.8,0.0,0.0
A&D 0.8,0.0,0.0	SPH basePied02
CON capsule	RZ 240
T 2.034,0,-0.25	T 2.034,0,-0.25
RY -26.565	RY -26.565
U 3	U 3
S 1.5,1.5,3	S 1.5,1.5,3
A&D 0.8,0.0,0.0	A&D 0.8,0.0,0.0
CON pied01	CON pied03
T 0,0,-3	T 0,0,-3
S 1.5,1.5,3	S 1.5,1.5,3
A&D 1.0,1.0,1.0	A&D 1.0,1.0,1.0
CYL tubePied01	CYL tubePied03
T 0,0,-3	T 0,0,-3
S 1.5,1.5,1.5	S 1.5,1.5,1.5
A&D 0.8,0.0,0.0	A&D 0.8,0.0,0.0
SPH basePied01	SPH basePied03



glDraw Method

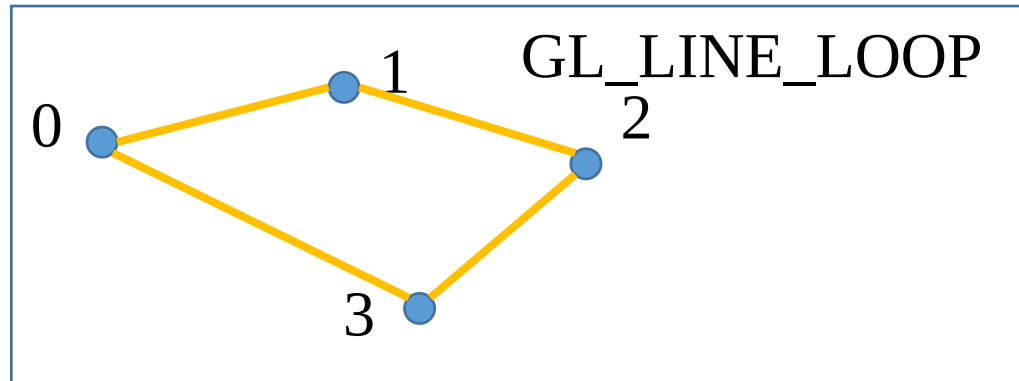
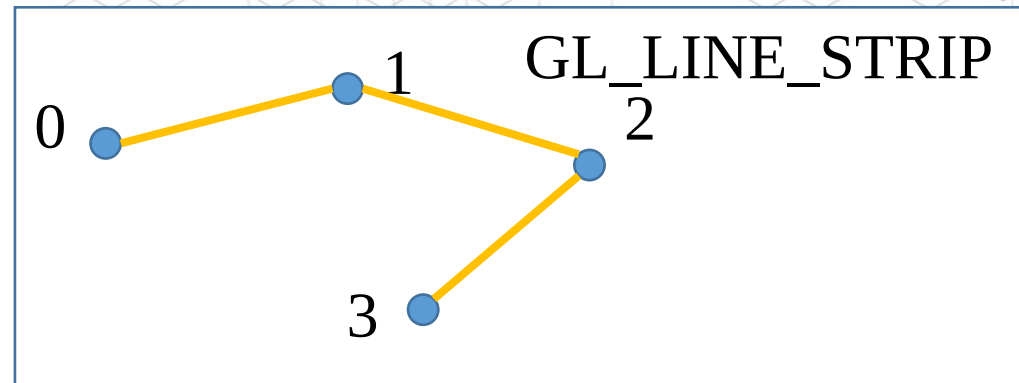
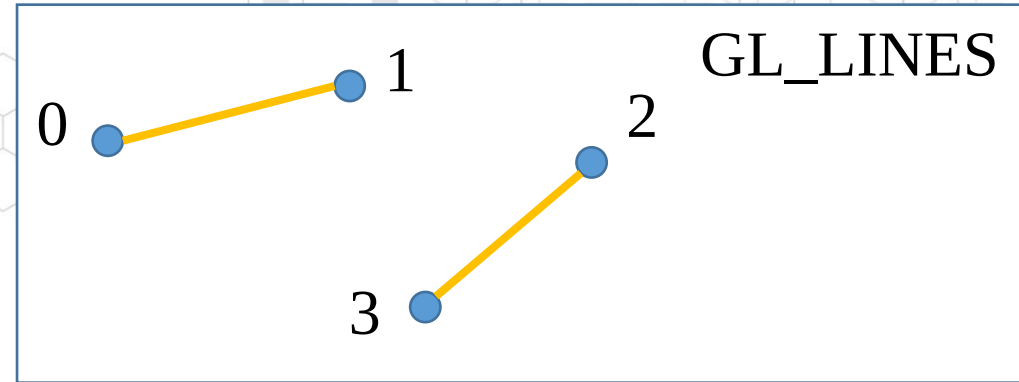
- Problème :
 - Transmettre les transformations géométriques pour dessiner les objets
 - Construction du produit matriciel de transformations géométriques
 - Utilisation des outils OpenGL/GLut

```
glPushMatrix();  
glTranslatef(-1,0,-2.1);  
glScalef(5,1,0.1);  
glutSolidCube(1);  
glPopMatrix();  
  
glPushMatrix();  
glTranslatef(1,0,-2);  
glScalef(0.5,0.5,4);  
glutSolidCylinder(1,1,...);  
glPopMatrix();
```



Basic 2D functions

- OpenGL drawing tools
 - Line width `glLineWidth(3);`
 - Basic 2D shapes:
 - GL_LINES (2N vertices),
 - GL_LINE_STRIP (N vertices)
 - GL_LINE_LOOP (N vertices),
 - GL_TRIANGLE (3N vertices),
 - GL_QUADS (4N vertices),
 - GL_POLYGON (N vertices)



Geometrical transformations

- Translation $T(x, y)$ of the futur drawing

```
glTranslatef(cx,cy,0.0f);
```

- Rotation $R(\alpha)$ of the futur drawing (alpha in degree)

```
glRotatef(alpha,0.0f,0.0f,1.0f);
```

- Scaling $S(sx, sy)$ of the futur drawing

```
glScalef(sx,sy,1.0f);
```

- Rotate then translate:

```
glPushMatrix();  
glTranslatef(cx,cy,0.0f);  
glRotatef(alpha,0.0f,0.0f,1.0f);  
Drawing  
glPopMatrix();
```

Drawing a circle?

```
glPushMatrix();  
glTranslatef(cx,cy,0.0f);  
glBegin(GL_LINE_LOOP);  
    for (int j=0; j<Npts; j++) {  
        double a = 2.0*j*M_PI/Npts;  
        glVertex2d(pointSize*cos(a),pointSize*sin(a));  
    }  
glEnd();  
glPopMatrix();
```

